

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект "Информационно-аналитическая система для РИО вуза".

Разработка сервиса "Методические указания"

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

М.Е. Петров

(инициалы, фамилия)

Группа ПО-226

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Петров М.Е., шифр 22-06-0019, группа ПО-226

1. Тема «Бизнес-проект "Информационно-аналитическая система для РИО вуза". Разработка сервиса "Методические указания"» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1585-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) Провести анализ деятельности редакционно-издательского отдела вуза и выявить проблематику традиционного подхода к проверке учебно-методических материалов;

2) Разработать концептуальную модель информационно-аналитической системы;

3) Спроектировать архитектуру системы, включая базу данных и веб-интерфейс;

4) Реализовать систему средствами веб-технологий (Python, FastAPI, PostgreSQL, HTML, CSS, JavaScript);

5) Разработать модуль автоматической проверки оформления документов формата DOCX (шрифт, отступы, интервалы, нумерация страниц);

6) Разработать модуль генерации титульных листов для методических указаний, учебных пособий и монографий.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: файлы формата DOCX (методические указания, учебные пособия, монографии), параметры для генерации титульных листов (название, автор, рецензенты, УДК, ISBN, описание), учётные данные пользователей (ФИО, email, пароль, факультет, кафедра, роль).

2) Выходными данными являются: результаты автоматической проверки документа (список ошибок с указанием страниц), исправленный документ с визуально выделенными ошибками, PDF-отчёт о проверке, сгенерированный титульный лист в формате DOCX, статистические отчёты по загруженным документам, JWT-токены для авторизации, подтверждение отправки файлов на электронную почту РИО.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области.

4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.4. Технический проект: общие сведения о программной системе, проект данных программной системы (ER-модель, реляционная модель), проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.5. Рабочий проект: маршруты приложения, спецификация контроллеров, системное тестирование разработанного веб-сайта.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

М.Е. Петров

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 102 страницам. Работа содержит 11 иллюстраций, 9 таблиц и 54 библиографических источников. Количество приложений – 1.

Перечень ключевых слов: Python, REST API, FastAPI, PostgreSQL, информационно-аналитическая система, автоматизация документооборота, проверка учебно-методических материалов, веб-приложение, генерация титульных листов, PDF-отчёты.

Объектом разработки является информационно-аналитическая система автоматизации проверки и обработки учебно-методических материалов редакционно-издательского отдела.

Цель работы – разработка информационно-аналитической системы для автоматизации процессов проверки, обработки и сопровождения учебно-методических документов с использованием Python и веб-технологий.

Задачи работы:

1. Изучить организацию работы редакционно-издательского отдела и процессы проверки учебно-методических материалов.
2. Проанализировать существующие подходы и средства автоматизации документооборота.
3. Разработать архитектуру информационно-аналитической системы.
4. Реализовать серверную часть с использованием FastAPI и PostgreSQL.
5. Разработать клиентскую часть веб-приложения.
6. Реализовать механизмы проверки документов, генерации титульных листов и формирования PDF-отчётов.
7. Провести тестирование разработанной системы.

При реализации использованы язык Python, веб-фреймворк FastAPI, СУБД PostgreSQL и современные средства веб-разработки.

Разработанная система может быть использована для автоматизации процессов проверки и обработки учебно-методических материалов.

ABSTRACT

The volume of work is 102 pages. The work contains 11 illustrations, 5 tables and 54 bibliographic sources. The number of applications is 1.

Keywords: Python, REST API, FastAPI, PostgreSQL, information and analytical system, workflow automation, verification of teaching materials, web application, cover page generation, PDF reports.

The object of development is an information and analytical automation system for checking and processing educational and methodological materials of the editorial and publishing department.

The purpose of the work is to develop an information and analytical system for automating the processes of checking, processing and maintaining educational documents using Python and web technologies.

Objectives:

1. Study the specifics of the editorial and publishing department work and the processes of checking educational materials.
2. Analyze existing approaches and tools for document management automation.
3. Develop the architecture of the information and analytical system.
4. Implement the server side using FastAPI and PostgreSQL.
5. Develop the client side of a web application.
6. Implement mechanisms for document verification, cover page generation and PDF report generation.
7. Test the developed system.

The software product was implemented using Python, FastAPI, PostgreSQL and modern web development tools.

The developed system can be used to automate the processes of verification and processing of educational materials.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	11
1 Анализ предметной области	13
1.1 Характеристика деятельности редакционно-издательского отдела вуза	13
1.2 Проблематика традиционного подхода к организации проверки материалов	14
1.3 Обоснование необходимости автоматизации	15
1.4 Анализ существующих систем автоматизации документооборота и проверки документов	16
1.4.1 Microsoft Word и встроенные средства рецензирования	16
1.4.2 Google Docs	16
1.4.3 Системы электронного документооборота	17
1.4.4 Вывод по анализу аналогов	17
1.5 Постановка задач на разработку	18
2 Техническое задание	19
2.1 Основание для разработки	19
2.2 Цель и назначение разработки	19
2.3 Требования к функциональности	19
2.4 Требования к интерфейсу	21
2.5 Требования к обработке ошибок	24
2.6 Требования к удобству использования (UX)	25
2.7 Моделирование вариантов использования	25
2.7.1 Вариант использования «Вход в систему»	28
2.7.2 Вариант использования «Выход из системы»	29
2.7.3 Вариант использования «Загрузка документа на проверку»	29
2.7.4 Вариант использования «Автоматическая проверка документа»	30
2.7.5 Вариант использования «Получение отчёта об ошибках»	30
2.7.6 Вариант использования «Скачивание документа с выделенными ошибками»	31

2.7.7	Вариант использования «Автоматическое сохранение корректного документа»	31
2.7.8	Вариант использования «Просмотр архива документов»	32
2.7.9	Вариант использования «Формирование титульного листа»	32
2.7.10	Вариант использования «Просмотр статистики»	33
2.7.11	Вариант использования «Управление пользователями»	33
2.7.12	Вариант использования «Настройка параметров системы»	33
2.8	Требования к надёжности	34
2.9	Требования к информационной безопасности	35
2.10	Требования к программной и технической совместимости	35
2.11	Требования к оформлению документации	36
3	Технический проект	37
3.1	Архитектура программной системы	37
3.2	Модуль обработки учебно-методических документов	39
3.2.1	Общая архитектура модуля	39
3.2.2	Интеграция с Microsoft Word через COM-интерфейс	39
3.2.3	Алгоритм автоматической проверки оформления	40
3.2.4	Формирование исправленной копии документа	44
3.2.5	Генерация PDF-отчёта	44
3.2.6	Генерация титульных листов	45
3.2.7	Управление сессиями и обновление токенов	46
3.2.8	Взаимодействие с базой данных через ORM	47
3.2.9	Предотвращение дублирования	48
3.2.10	Обработка ошибок и восстановление	49
3.2.11	Хранение данных	49
3.2.12	Логирование	50
3.2.13	Заключение	50
3.3	База данных и её проектирование	51
3.3.1	ER-модель данных	51
3.3.2	Нормализация	53
3.3.3	Реляционная модель данных	53

3.3.4	Описание структуры базы данных	55
4	Рабочий проект	58
4.1	Маршруты приложения	58
4.2	Спецификация контроллеров приложения	59
4.2.1	Контроллер модуля проверки документов	59
4.2.2	Контроллер модуля генерации титульных листов	60
4.2.3	Контроллер модуля отправки в РИО	60
4.3	Системное тестирование разработанного web-сайта	61
4.3.1	Тестирование модуля проверки документов	61
4.3.2	Тестирование модуля генерации титульных листов	62
4.3.3	Тестирование модуля отправки в РИО	63
4.3.4	Результаты тестирования	63
	ЗАКЛЮЧЕНИЕ	65
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
	ПРИЛОЖЕНИЕ А Фрагменты исходного кода программы	72
	На отдельных листах (CD-RW в прикрепленном конверте)	102

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

РП — рабочий проект;

ТЗ — техническое задание;

ТП — технический проект;

JPEG (Joint Photographic Experts Group) — растровый графический формат изображений и фотографий с высокой степенью сжатия;

DCT (Discrete Cosine Transform) — дискретное косинусное преобразование, используемое в алгоритме сжатия JPEG;

RGB (Red, Green, Blue) — цветовая модель, основанная на представлении цвета с использованием красного, зелёного и синего компонентов;

YCbCr — цветовое пространство, включающее яркостную компоненту (Y) и две цветоразностные компоненты (Cb, Cr);

GUI (Graphical User Interface) — графический пользовательский интерфейс;

API (Application Programming Interface) — программный интерфейс взаимодействия компонентов системы;

IDCT (Inverse Discrete Cosine Transform) — обратное дискретное косинусное преобразование;

ROI (Region of Interest) — область интереса изображения, используемая при анализе и обработке данных.

ВВЕДЕНИЕ

В условиях роста объёмов учебно-методической документации особую актуальность приобретает автоматизация процессов проверки, оформления и сопровождения документов. В образовательных организациях значительное количество времени затрачивается на ручную проверку учебных пособий, методических указаний и иных материалов на соответствие установленным требованиям оформления. Использование традиционного подхода приводит к увеличению трудозатрат сотрудников редакционно-издательских отделов, повышает вероятность появления ошибок и усложняет контроль качества документации.

Современные информационные технологии позволяют автоматизировать процессы обработки документов, обеспечить централизованное хранение информации и повысить эффективность взаимодействия между пользователями системы. Особую значимость приобретают веб-ориентированные информационно-аналитические системы, предоставляющие возможность автоматизированной проверки документов, генерации отчётов, формирования титульных листов и организации взаимодействия между авторами и сотрудниками редакционно-издательского отдела.

В рамках данной выпускной квалификационной работы рассматривается разработка информационно-аналитической системы автоматизации проверки учебно-методических материалов. Особое внимание уделяется реализации механизмов анализа параметров форматирования документов, автоматической генерации титульных листов, формированию PDF-отчётов о выявленных нарушениях, а также организации клиент-серверного взаимодействия.

Целью выпускной квалификационной работы является разработка информационно-аналитической системы для автоматизации процессов проверки и обработки учебно-методических документов редакционно-издательского отдела.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить особенности организации работы редакционно-издательского отдела и процессов проверки документов;
- проанализировать существующие средства автоматизации документооборота и проверки документации;
- разработать архитектуру информационно-аналитической системы;
- реализовать серверную часть системы с использованием FastAPI и PostgreSQL;
- разработать клиентскую часть веб-приложения;
- реализовать механизмы проверки документов и анализа параметров форматирования;
- разработать модули генерации титульных листов и формирования PDF-отчётов;
- провести тестирование разработанной системы и оценить корректность её работы.

1 Анализ предметной области

1.1 Характеристика деятельности редакционно-издательского отдела вуза

Редакционно-издательский отдел (РИО) высшего учебного заведения является структурным подразделением, обеспечивающим подготовку, проверку, оформление и выпуск учебно-методической, научной и справочной литературы, используемой в образовательном процессе. К числу основных материалов, поступающих в РИО, относятся методические указания, учебные пособия, практикумы, монографии, сборники научных трудов и иные издания.

Современный образовательный процесс требует постоянного обновления учебно-методической базы. Изменение федеральных государственных образовательных стандартов, развитие научных направлений, внедрение новых технологий обучения и цифровых платформ обуславливают регулярную подготовку новых материалов преподавателями университета.

РИО выполняет функции организационного и нормативного сопровождения издательской деятельности вуза. Сотрудники подразделения принимают материалы от авторов, проверяют их соответствие установленным требованиям оформления, контролируют наличие обязательных структурных элементов, подготавливают замечания, формируют титульные листы, а также ведут учёт поступивших и утверждённых работ.

В условиях большого количества авторов и поступающих документов особую значимость приобретают скорость обработки материалов, точность проверки и удобство взаимодействия между преподавателями и сотрудниками отдела. Эффективность деятельности РИО напрямую влияет на своевременное обеспечение образовательного процесса актуальными учебными материалами.

1.2 Проблематика традиционного подхода к организации проверки материалов

Традиционный подход к работе редакционно-издательского отдела, основанный на ручной обработке документов, сопровождается рядом существенных недостатков, снижающих эффективность работы подразделения:

1. Высокая трудоёмкость проверки оформления. В большинстве случаев сотрудники РИО вручную проверяют поступившие документы на соответствие установленным требованиям: параметры шрифта, интервалы, отступы, структура заголовков, оформление списков литературы, таблиц, рисунков и других элементов. При большом количестве работ такой процесс занимает значительное время.

2. Высокая вероятность человеческих ошибок. Ручная проверка не гарантирует обнаружение всех нарушений оформления. Из-за большого объёма текста, однотипности операций и человеческого фактора часть ошибок может остаться незамеченной, что приводит к возврату материалов на доработку уже на поздних этапах.

3. Длительное взаимодействие с авторами. После выявления замечаний сотрудник вынужден вручную составлять перечень ошибок либо вносить исправления в режиме рецензирования документа. Это увеличивает сроки согласования и повторной подачи материалов.

4. Отсутствие централизованного хранилища работ. Готовые и поступившие материалы часто хранятся в отдельных папках, на персональных компьютерах или сетевых дисках. Это затрудняет поиск ранее утверждённых изданий, ведение архива и получение статистических сведений.

5. Сложность формирования отчётности. Для подготовки отчётов по количеству проверенных работ, числу авторов, срокам обработки и видам материалов требуется вручную собирать данные из различных источников. Это увеличивает нагрузку на сотрудников и снижает оперативность получения информации.

6. Отсутствие унифицированных шаблонов. Авторы нередко готовят титульные листы и иные элементы документа самостоятельно, что приводит к многочисленным отклонениям от утверждённых образцов и повторным исправлениям.

Таким образом, ручной подход к организации работы РИО не обеспечивает необходимой скорости, точности и удобства обработки документов в современных условиях.

1.3 Обоснование необходимости автоматизации

Решение перечисленных проблем возможно путём внедрения специализированной информационно-аналитической системы для автоматизации деятельности редакционно-издательского отдела вуза.

Разрабатываемая система должна обеспечить:

- загрузку пользователями учебно-методических и научных материалов через веб-интерфейс;
- автоматическую проверку документов на соответствие требованиям оформления;
- формирование подробного отчёта с перечнем найденных ошибок;
- автоматическое создание файла формата DOCX с выделенными нарушениями;
- автоматическое сохранение корректно оформленных работ в централизованном хранилище;
- ведение статистики поступивших, проверенных и утверждённых материалов;
- генерацию титульных листов по утверждённым шаблонам;
- разграничение прав доступа между авторами, редакторами и администраторами системы;
- удобный интерфейс взаимодействия пользователей с системой.

Автоматизация позволит существенно сократить время проверки материалов, снизить количество ошибок, ускорить процесс согласования документов и обеспечить прозрачный контроль деятельности подразделения.

1.4 Анализ существующих систем автоматизации документооборота и проверки документов

На современном рынке представлены различные программные решения, связанные с управлением документами, совместной работой и проверкой текстовых материалов. Однако большинство из них не ориентировано на специфику работы редакционно-издательского отдела вуза.

1.4.1 Microsoft Word и встроенные средства рецензирования

Текстовый редактор Microsoft Word широко применяется для подготовки и проверки документов. Он предоставляет следующие возможности:

- создание и редактирование документов;
- проверка орфографии и грамматики;
- режим рецензирования и внесения исправлений;
- использование шаблонов оформления.

Недостатком данного подхода является отсутствие автоматизированной проверки документа на соответствие локальным требованиям конкретного вуза. Проверка структуры, параметров оформления и соответствия внутренним стандартам выполняется вручную.

1.4.2 Google Docs

Google Docs является облачным сервисом для совместной работы с документами. Его возможности включают:

- онлайн-редактирование файлов;
- совместную работу нескольких пользователей;
- систему комментариев и предложений;
- хранение документов в облаке.

Несмотря на удобство совместного редактирования, сервис не предоставляет специализированных механизмов автоматической проверки учебно-методических материалов по требованиям университета, а также не содержит функций аналитической отчётности для РИО.

1.4.3 Системы электронного документооборота

Корпоративные системы электронного документооборота предназначены для регистрации, маршрутизации и хранения документов. Они позволяют:

- вести архив документов;
- организовывать согласование файлов;
- разграничивать права доступа;
- контролировать исполнение задач.

Однако подобные системы являются универсальными решениями и не включают инструменты интеллектуального анализа оформления документов DOCX, автоматического подчёркивания ошибок и генерации титульных листов.

1.4.4 Вывод по анализу аналогов

Проведённый анализ существующих решений показал, что универсальные офисные продукты и системы электронного документооборота способны решать лишь отдельные задачи редакционно-издательского отдела.

Одни системы обеспечивают редактирование документов, другие — хранение файлов и организацию согласования, однако ни одно из рассмотренных решений не сочетает в себе функции автоматической проверки оформления методических материалов, генерации отчётов об ошибках, формирования исправленного DOCX-файла и накопления статистики работы подразделения.

Следовательно, наиболее целесообразным является создание собственной специализированной информационно-аналитической системы, учитывающей особенности документооборота и нормативные требования конкретного высшего учебного заведения.

1.5 Постановка задач на разработку

На основании проведённого анализа были сформулированы следующие задачи, которые должна решать разрабатываемая система:

1. Обеспечить регистрацию и авторизацию пользователей с разграничением прав доступа.
2. Реализовать загрузку методических, учебных и научных материалов в формате DOCX.
3. Выполнить автоматическую проверку документов на соответствие установленным требованиям оформления.
4. Формировать отчёт с перечнем найденных нарушений.
5. Создавать копию документа DOCX с визуальным выделением ошибок.
6. Автоматически сохранять корректно оформленные работы в централизованной базе данных.
7. Реализовать хранение и поиск ранее загруженных материалов.
8. Предоставить средства формирования статистической и аналитической отчётности.
9. Реализовать модуль автоматического создания титульных листов по шаблону вуза.
10. Создать удобный веб-интерфейс для пользователей и административную панель для управления системой.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Информационно-аналитическая система для РИО вуза».

2.2 Цель и назначение разработки

Основной целью работы является разработка и внедрение информационно-аналитической системы для автоматизации процессов проверки, хранения и сопровождения учебно-методических и научных материалов, поступающих в редакционно-издательский отдел вуза.

Задачами разработки являются:

- анализ предметной области и выявление требований к разрабатываемой системе;
- проектирование архитектуры программного средства, базы данных и пользовательского интерфейса;
- разработка серверной и клиентской частей информационной системы;
- реализация и интеграция модулей автоматической проверки документов, хранения файлов и формирования отчётности;
- тестирование, внедрение и оценка эффективности разработанной системы.

2.3 Требования к функциональности

Информационная система должна обеспечивать выполнение следующих функций:

1. **Управление пользователями.** Система должна обеспечивать регистрацию и авторизацию пользователей по логину и паролю. Должно быть реализовано разграничение прав доступа: администратор, редактор, пользо-

ватель. Администратор должен иметь возможность управления учётными записями пользователей.

2. Загрузка документов. Пользователь должен иметь возможность загружать методические материалы, учебные пособия, монографии и иные документы в формате DOCX через веб-интерфейс системы.

3. Автоматическая проверка оформления. После загрузки документа система должна автоматически анализировать файл на соответствие установленным требованиям оформления. Проверка должна включать контроль структуры документа, параметров шрифта, интервалов, отступов, заголовков, таблиц, изображений, списков литературы и иных элементов.

4. Формирование отчёта об ошибках. По завершении анализа система должна формировать отчёт, содержащий перечень обнаруженных нарушений, описание ошибок и рекомендации по их устранению.

5. Создание документа с выделением ошибок. Система должна создавать копию загруженного документа в формате DOCX, в которой найденные ошибки визуально выделены средствами форматирования.

6. Хранение корректных документов. Если загруженный документ соответствует всем требованиям, он должен автоматически сохраняться в централизованной базе данных или файловом архиве системы.

7. Поиск и просмотр документов. Система должна обеспечивать просмотр списка ранее загруженных материалов, поиск по названию, автору, дате загрузки, статусу проверки и другим параметрам.

8. Генерация титульных листов. Пользователь должен иметь возможность заполнить форму с исходными данными, после чего система должна автоматически сформировать титульный лист по утверждённому шаблону вуза.

9. Статистика и аналитика. Система должна формировать сводные данные по количеству загруженных документов, числу успешных проверок, количеству найденных ошибок, активности пользователей и временным показателям обработки материалов.

10. Журналирование действий. Система должна сохранять сведения о действиях пользователей: загрузка файлов, проверка документов, вход в систему, генерация титульных листов и иные операции.

2.4 Требования к интерфейсу

Интерфейс информационной системы представляет собой веб-приложение с единым современным стилем оформления, обеспечивающее удобную и интуитивно понятную навигацию. Пользовательский интерфейс должен быть адаптирован для работы в распространённых браузерах и обеспечивать быстрый доступ к основным функциям системы.

Основные элементы интерфейса включают:

- главную страницу системы;
- форму авторизации и регистрации пользователей;
- личный кабинет пользователя;
- страницу загрузки документов на проверку;
- страницу просмотра результатов проверки;
- страницу архива сохранённых документов;
- страницу генерации титульных листов;
- административную панель управления пользователями;
- модуль просмотра статистики и аналитических данных.

На рисунке 2.1 представлена страница авторизации системы.

Как показано на рисунке, форма авторизации содержит поля ввода логина и пароля, кнопку входа в систему, а также элементы перехода к регистрации нового пользователя. После успешной авторизации пользователь получает доступ к функциональным возможностям системы в соответствии со своей ролью.

На рисунке 2.2 представлен внешний вид личного кабинета пользователя.

Рисунок 2.1 – Страница авторизации

Рисунок 2.2 – Личный кабинет пользователя

Личный кабинет содержит навигационное меню, сведения о текущем пользователе, основные разделы системы, а также быстрый доступ к загрузке документов, архиву файлов и генерации титульных листов.

На рисунке 2.3 представлена страница проверки документов.

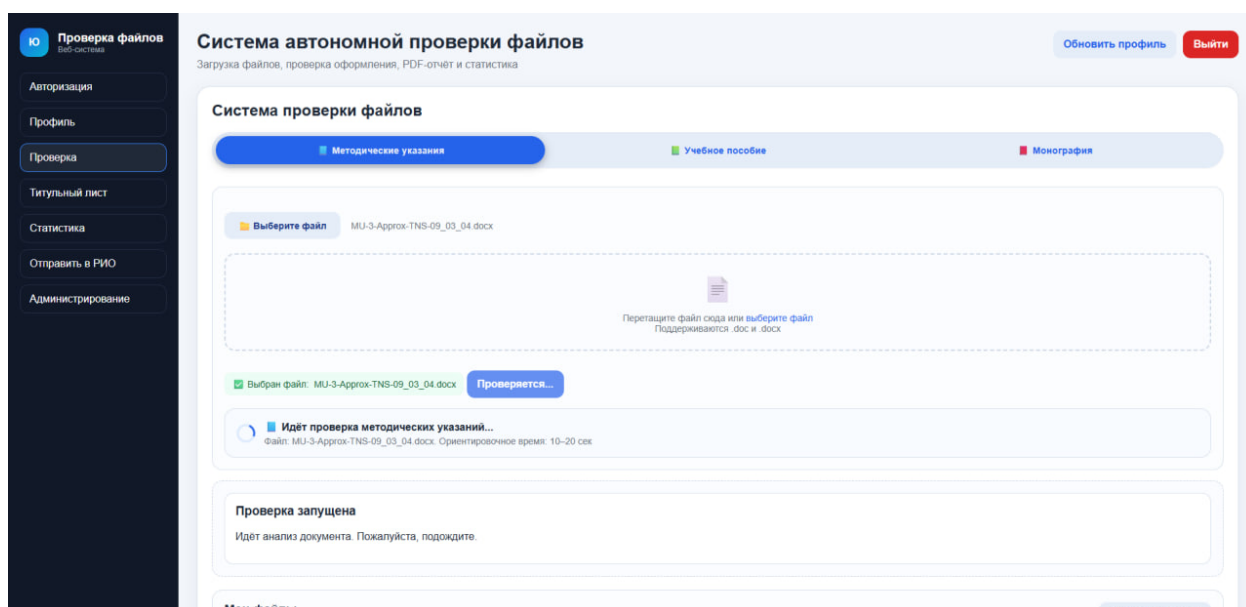


Рисунок 2.3 – Страница проверки документов

Данная страница содержит форму выбора файла формата DOCX, кнопку запуска автоматической проверки, область отображения текущего статуса обработки и результаты анализа после завершения проверки.

На рисунке 2.4 представлена страница генерации титульных листов.

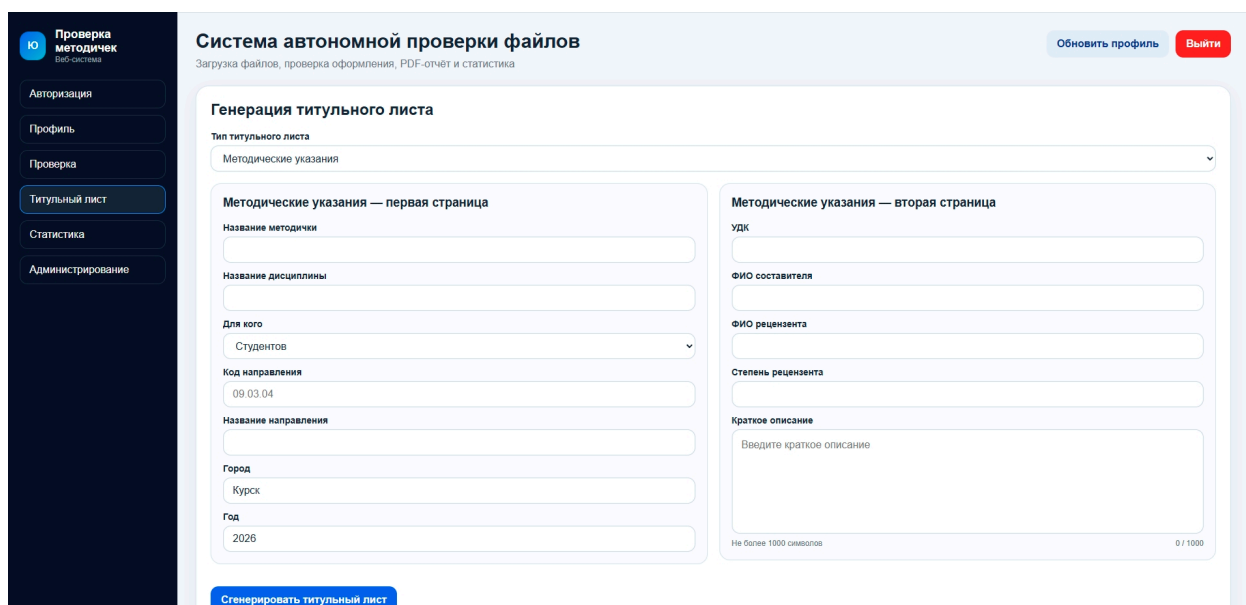


Рисунок 2.4 – Страница генерации титульных листов

Как показано на рисунке, страница содержит форму ввода реквизитов работы: наименование дисциплины, тема работы, сведения об авторе, руко-

водителе и кафедре. После заполнения формы пользователь может сформировать готовый документ в формате DOCX.

На рисунке 2.5 представлена страница статистики и аналитики.

Рисунок 2.5 – Страница статистики

Страница статистики содержит сведения о количестве зарегистрированных пользователей, числе загруженных документов, результатах проверок, количестве обнаруженных ошибок, а также иные аналитические показатели деятельности системы.

Интерфейс системы должен обеспечивать логичное расположение элементов управления, единообразие оформления страниц, наглядность представления результатов работы и минимальное количество действий пользователя при выполнении основных операций.

2.5 Требования к обработке ошибок

Система должна обеспечивать корректную обработку исключительных ситуаций.

Основные требования:

- при возникновении ошибки пользователю должно отображаться понятное сообщение;

- внутренние ошибки системы не должны раскрывать служебную информацию;
- ошибки должны фиксироваться в журнале событий;
- должна быть реализована обработка:
 - некорректных входных данных;
 - ошибок загрузки файлов;
 - ошибок обработки документов;
 - ошибок взаимодействия с базой данных.

2.6 Требования к удобству использования (UX)

Система должна обеспечивать удобство и простоту взаимодействия пользователей.

Основные требования:

- интерфейс должен быть интуитивно понятным;
- количество действий для выполнения основной операции (загрузка и проверка документа) должно быть минимальным;
- результаты проверки должны отображаться в структурированном виде;
- должна быть реализована визуальная обратная связь (индикаторы загрузки, статусы операций).

2.7 Моделирование вариантов использования

Для определения функциональных возможностей разрабатываемой системы и взаимодействия пользователей с её компонентами была выполнена модель вариантов использования.

В системе предусмотрены следующие роли пользователей:

Администратор — обладает полными правами доступа, включая управление пользователями, просмотр статистики, настройку системы, удаление документов и контроль работы пользователей.

Редактор — осуществляет просмотр загруженных материалов, контроль результатов автоматической проверки, работу с архивом документов и формирование отчётности.

Пользователь — выполняет загрузку документов, получает результаты проверки, скачивает отчёты, использует модуль генерации титульных листов и просматривает историю своих материалов.

Основные прецеденты (варианты использования) системы:

1. Вход в систему.
2. Регистрация пользователя.
3. Загрузка документа на проверку.
4. Автоматическая проверка оформления.
5. Просмотр отчёта об ошибках.
6. Скачивание исправленного документа.
7. Сохранение корректного документа в архив.
8. Поиск и просмотр ранее загруженных работ.
9. Генерация титульного листа.
10. Просмотр статистики работы системы.
11. Управление пользователями.
12. Выход из системы.

Для повышения наглядности диаграмма прецедентов была разделена по ролям пользователей. Отдельно представлены варианты использования для пользователя, администратора и редактора.

На рисунке 2.6 представлена диаграмма прецедентов пользователя.

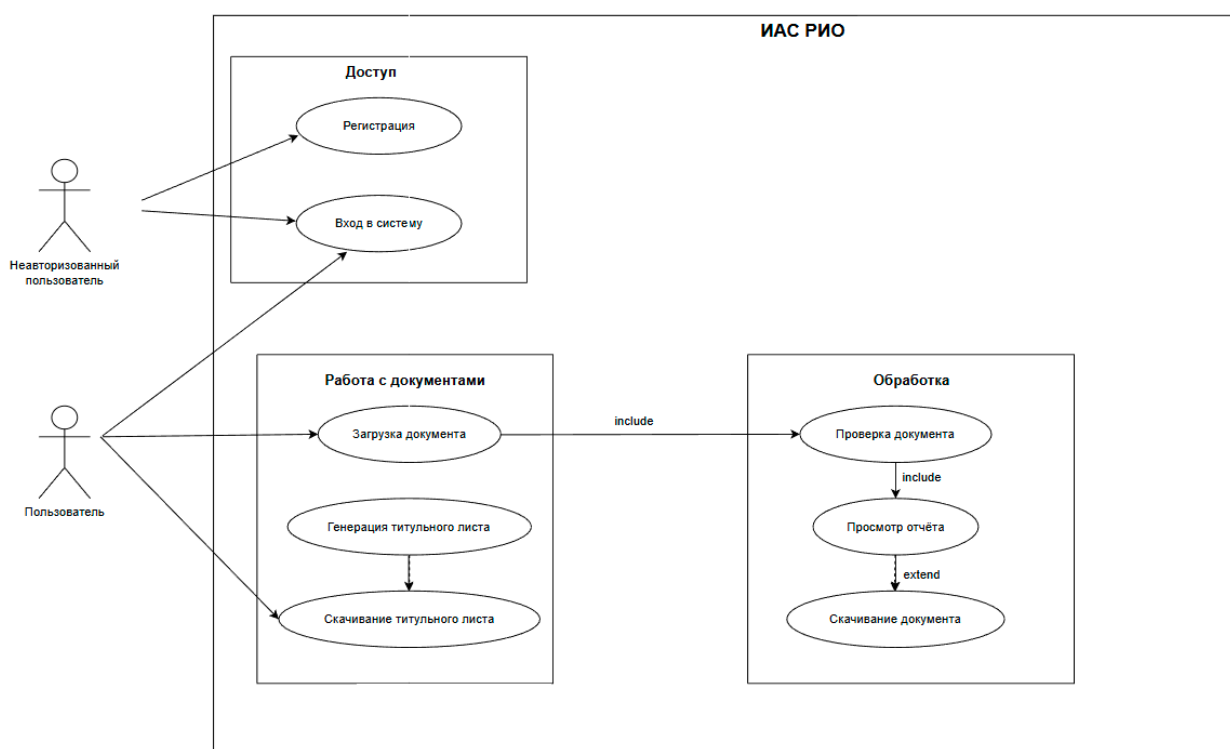


Рисунок 2.6 – Диаграмма прецедентов пользователя

На рисунке 2.7 представлена диаграмма прецедентов администратора.

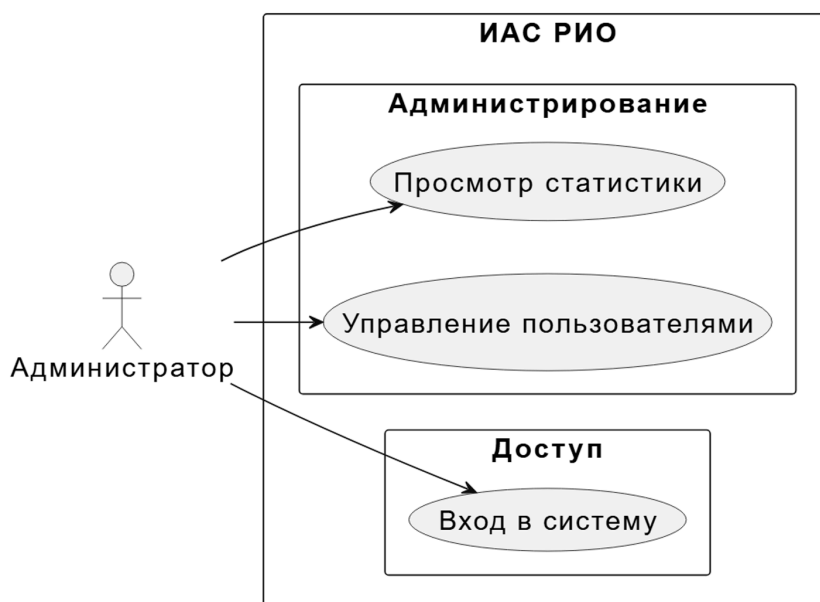


Рисунок 2.7 – Диаграмма прецедентов администратора

На рисунке 2.8 представлена диаграмма прецедентов редактора.

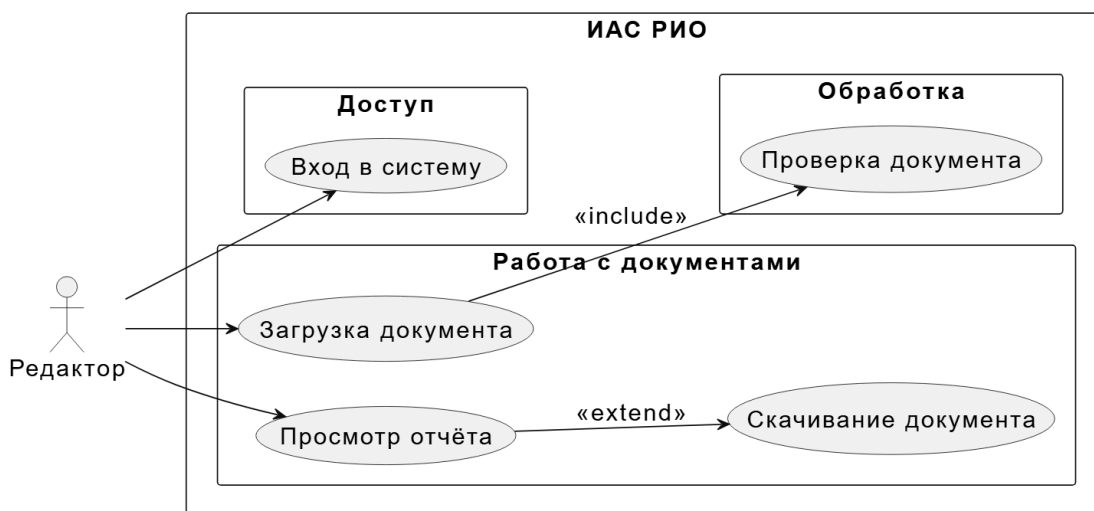


Рисунок 2.8 – Диаграмма прецедентов редактора

2.7.1 Вариант использования «Вход в систему»

Заинтересованные лица и их требования: пользователь желает получить доступ к функциям системы путём ввода логина и пароля.

Предусловие: пользователь не авторизован; открыта страница входа.

Постусловие: пользователь успешно авторизован и перенаправлен в личный кабинет либо на главную страницу системы.

Основной успешный сценарий:

1. Пользователь открывает страницу входа.
2. Вводит логин и пароль.
3. Нажимает кнопку «Войти».
4. Система выполняет поиск пользователя в базе данных.
5. Выполняется проверка пароля.
6. Создаётся пользовательская сессия.
7. Пользователь получает доступ к функциям системы.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит неверный логин или пароль.
2. Система отображает сообщение об ошибке авторизации.
3. Пользователь остаётся на странице входа.

Альтернативный сценарий (пустые поля):

1. Пользователь не заполняет одно из обязательных полей.
2. Система выводит сообщение о необходимости заполнения формы.

2.7.2 Вариант использования «Выход из системы»

Заинтересованные лица и их требования: авторизованный пользователь желает завершить работу с системой.

Предусловие: пользователь авторизован.

Постусловие: пользовательская сессия завершена.

Основной успешный сценарий:

1. Пользователь нажимает кнопку «Выход».
2. Система удаляет активную сессию.
3. Пользователь перенаправляется на страницу входа.

2.7.3 Вариант использования «Загрузка документа на проверку»

Заинтересованные лица и их требования: пользователь желает отправить документ в формате DOCX для автоматической проверки.

Предусловие: пользователь авторизован.

Постусловие: файл загружен в систему и передан модулю проверки.

Основной успешный сценарий:

1. Пользователь открывает страницу загрузки.
2. Выбирает файл формата DOCX.
3. Нажимает кнопку «Загрузить».
4. Система сохраняет файл во временное хранилище.
5. Запускается процедура проверки документа.

Альтернативный сценарий (неподдерживаемый формат):

1. Пользователь выбирает файл иного формата.
2. Система выводит сообщение об ошибке.
3. Загрузка не выполняется.

Альтернативный сценарий (файл не выбран):

1. Пользователь нажимает кнопку загрузки без выбора файла.

2. Система предлагает выбрать документ.

2.7.4 Вариант использования «Автоматическая проверка документа»

Заинтересованные лица и их требования: пользователь желает получить результат проверки оформления документа.

Предусловие: документ успешно загружен.

Постусловие: сформирован результат проверки.

Основной успешный сценарий:

1. Система анализирует структуру документа.
2. Выполняется проверка параметров оформления.
3. Выявляются ошибки и несоответствия требованиям.
4. Формируется перечень найденных нарушений.

Альтернативный сценарий (ошибка обработки файла):

1. Документ повреждён либо содержит некорректную структуру.
2. Проверка прерывается.
3. Пользователь получает сообщение об ошибке обработки документа.

2.7.5 Вариант использования «Получение отчёта об ошибках»

Заинтересованные лица и их требования: пользователь желает ознакомиться с найденными нарушениями.

Предусловие: проверка завершена.

Постусловие: пользователю отображён отчёт.

Основной успешный сценарий:

1. После завершения проверки система формирует отчёт.
2. Пользователь открывает страницу результата.
3. Отображается список ошибок с пояснениями.

Альтернативный сценарий (ошибок не обнаружено):

1. Проверка завершена без замечаний.
2. Система отображает сообщение о корректности документа.

2.7.6 Вариант использования «Скачивание документа с выделенными ошибками»

Заинтересованные лица и их требования: пользователь желает скачать исправляемый файл с визуально отмеченными ошибками.

Предусловие: документ содержит нарушения.

Постусловие: пользователю предоставлен файл DOCX для скачивания.

Основной успешный сценарий:

1. Пользователь нажимает кнопку скачивания.
2. Система формирует копию документа.
3. Ошибки подчёркиваются или выделяются.
4. Пользователь скачивает файл.

Альтернативный сценарий (файл не сформирован):

1. Во время генерации файла возникает ошибка.
2. Система выводит сообщение о невозможности подготовки документа.

2.7.7 Вариант использования «Автоматическое сохранение корректного документа»

Заинтересованные лица и их требования: пользователь желает сохранить корректно оформленную работу в архиве системы.

Предусловие: ошибок в документе не обнаружено.

Постусловие: файл помещён в централизованное хранилище.

Основной успешный сценарий:

1. Проверка завершается без ошибок.
2. Система сохраняет документ в архив.
3. Информация о работе заносится в базу данных.

Альтернативный сценарий (ошибка сохранения):

1. Возникает ошибка доступа к хранилищу либо базе данных.
2. Система фиксирует ошибку в журнале событий.

3. Пользователю выводится уведомление.

2.7.8 Вариант использования «Просмотр архива документов»

Заинтересованные лица и их требования: пользователь желает просмотреть ранее сохранённые документы.

Предусловие: пользователь авторизован.

Постусловие: отображён список документов архива.

Основной успешный сценарий:

1. Пользователь открывает раздел архива.
2. Система загружает список документов.
3. Пользователь просматривает или скачивает нужный файл.

Альтернативный сценарий (архив пуст):

1. В системе отсутствуют сохранённые документы.
2. Отображается сообщение «Архив пуст».

2.7.9 Вариант использования «Формирование титульного листа»

Заинтересованные лица и их требования: пользователь желает автоматически создать титульный лист по шаблону вуза.

Предусловие: пользователь авторизован.

Постусловие: сформирован титульный лист.

Основной успешный сценарий:

1. Пользователь открывает форму генерации титульного листа.
2. Заполняет необходимые поля.
3. Нажимает кнопку формирования.
4. Система создаёт документ DOCX.

Альтернативный сценарий (не заполнены обязательные поля):

1. Пользователь оставляет обязательные поля пустыми.
2. Система сообщает о необходимости заполнения данных.

2.7.10 Вариант использования «Просмотр статистики»

Заинтересованные лица и их требования: администратор желает получить статистические сведения о работе системы.

Предусловие: администратор авторизован.

Постусловие: отображён аналитический отчёт.

Основной успешный сценарий:

1. Администратор открывает раздел статистики.
2. Система подсчитывает показатели.
3. Отображаются данные по количеству проверок, ошибок, сохранённых документов и пользователям.

Альтернативный сценарий (недостаточно прав):

1. Обычный пользователь пытается открыть раздел статистики.
2. Система запрещает доступ.

2.7.11 Вариант использования «Управление пользователями»

Заинтересованные лица и их требования: администратор желает управлять учётными записями пользователей.

Предусловие: администратор авторизован.

Постусловие: учётные записи обновлены.

Основной успешный сценарий:

1. Администратор открывает раздел пользователей.
2. Добавляет, редактирует или удаляет записи.
3. Система сохраняет изменения.

Альтернативный сценарий (дублирование логина):

1. При создании пользователя введён уже существующий логин.
2. Система выводит сообщение об ошибке.

2.7.12 Вариант использования «Настройка параметров системы»

Заинтересованные лица и их требования: администратор желает изменить правила проверки документов и параметры работы системы.

Предусловие: администратор авторизован.

Постусловие: новые параметры сохранены.

Основной успешный сценарий:

1. Администратор открывает настройки.
2. Изменяет параметры.
3. Подтверждает сохранение.
4. Система применяет новые значения.

Альтернативный сценарий (некорректные параметры):

1. Введены недопустимые значения настроек.
2. Система отменяет сохранение и выводит сообщение об ошибке.

2.8 Требования к надёжности

Разрабатываемая информационно-аналитическая система должна обеспечивать стабильную и корректную работу при различных режимах эксплуатации, а также гарантировать сохранность пользовательских данных.

Целостность данных. Все операции загрузки документов, сохранения результатов проверки, регистрации пользователей и формирования отчётов должны выполняться атомарно. При возникновении ошибки операция должна быть отменена без повреждения существующих данных.

Устойчивость к сбоям. При аварийном завершении работы сервера, отключении электропитания или программном сбое система должна сохранять ранее записанные данные. После повторного запуска система должна продолжить работу в штатном режиме.

Контроль корректности входных данных. На уровне приложения должна выполняться проверка корректности вводимых пользователями данных: обязательность заполнения полей, допустимые форматы файлов, ограничения длины строковых значений, корректность дат и иных параметров.

Логирование ошибок. Критические ошибки, исключения и события работы системы должны фиксироваться в журнале событий. Записи журнала должны содержать дату, время, уровень важности и описание ошибки.

Резервное копирование. База данных и архив документов должны допускать резервное копирование штатными средствами используемой СУБД и файловой системы. Восстановление данных должно выполняться из резервной копии.

2.9 Требования к информационной безопасности

Система должна обеспечивать защиту данных от несанкционированного доступа и разграничение прав пользователей.

Авторизация пользователей. Доступ к функциям системы должен предоставляться только после успешного ввода логина и пароля.

Хранение паролей. Пароли пользователей не должны храниться в открытом виде. Для хранения необходимо использовать криптографическое хеширование.

Разграничение прав доступа. В системе должны поддерживаться роли пользователей:

- пользователь — загрузка документов, просмотр результатов, генерация титульных листов;
- администратор — полный доступ, включая управление пользователями и просмотр статистики.

Проверка доступа на серверной стороне. Права пользователя должны проверяться сервером при выполнении каждого защищённого действия.

Защита от SQL-инъекций. Доступ к базе данных должен осуществляться с использованием ORM либо параметризованных запросов.

Защита файлового хранилища. Загруженные документы и архивные материалы должны храниться в каталогах, недоступных для прямого неавторизованного обращения.

2.10 Требования к программной и технической совместимости

Система должна функционировать в типовой вычислительной среде и не предъявлять завышенных требований к аппаратным ресурсам.

Операционная система сервера:

- Windows 10/11.

Аппаратные требования:

- процессор с тактовой частотой от 2 ГГц;
- оперативная память не менее 4 ГБ;
- свободное дисковое пространство от 1 ГБ без учёта пользовательских файлов.

Клиентская часть. Для работы пользователей необходимы текстовый редактор Word и современный веб-браузер:

- Яндекс Браузер;
- Google Chrome;
- Mozilla Firefox;
- Microsoft Edge.

2.11 Требования к оформлению документации

Разработка программной документации и программного изделия должна выполняться в соответствии с требованиями действующих нормативных документов ГОСТ 19.102–77 и ГОСТ 34.601–90. Единая система программной документации.

3 Технический проект

3.1 Архитектура программной системы

Разрабатываемая информационно-аналитическая система построена по архитектуре «клиент-сервер» с использованием REST-подхода.

Серверная часть реализована на языке Python с использованием фреймворка FastAPI и обеспечивает обработку запросов, выполнение бизнес-логики, а также взаимодействие с базой данных PostgreSQL и файловым хранилищем документов.

Клиентская часть представляет собой веб-приложение, доступ к которому осуществляется через браузер. Взаимодействие между клиентом и сервером выполняется по протоколу HTTP с использованием формата JSON для обмена данными.

Архитектура системы включает следующие основные компоненты:

- клиентское веб-приложение (интерфейс пользователя);
- серверное приложение (FastAPI);
- система управления базами данных PostgreSQL;
- файловое хранилище для загружаемых и сгенерированных документов;
- модуль анализа и проверки документов формата DOCX.

Логика работы системы организована следующим образом. Пользователь через веб-интерфейс выполняет действия (авторизация, загрузка документа, запуск проверки и др.), которые инициируют HTTP-запросы к серверу. Сервер обрабатывает запрос, при необходимости обращается к базе данных и файловой системе, выполняет проверку документа и возвращает результат клиенту.

Модуль проверки документов осуществляет анализ структуры и оформления файлов формата DOCX, выявляет несоответствия установленным требованиям и формирует отчёт об ошибках. При необходимости система генерирует исправленный файл с визуальным выделением нарушений.

Все загружаемые документы сохраняются в файловой системе, а информация о них (путь к файлу, результаты проверки, пользователь и др.) — в базе данных PostgreSQL. Это обеспечивает разделение данных и повышает масштабируемость системы.

Для обеспечения безопасности и управления доступом используется механизм аутентификации пользователей. После успешного входа пользователь получает токен доступа, который используется при последующих запросах к API.

Общая схема взаимодействия компонентов системы представлена на рисунке 3.1.

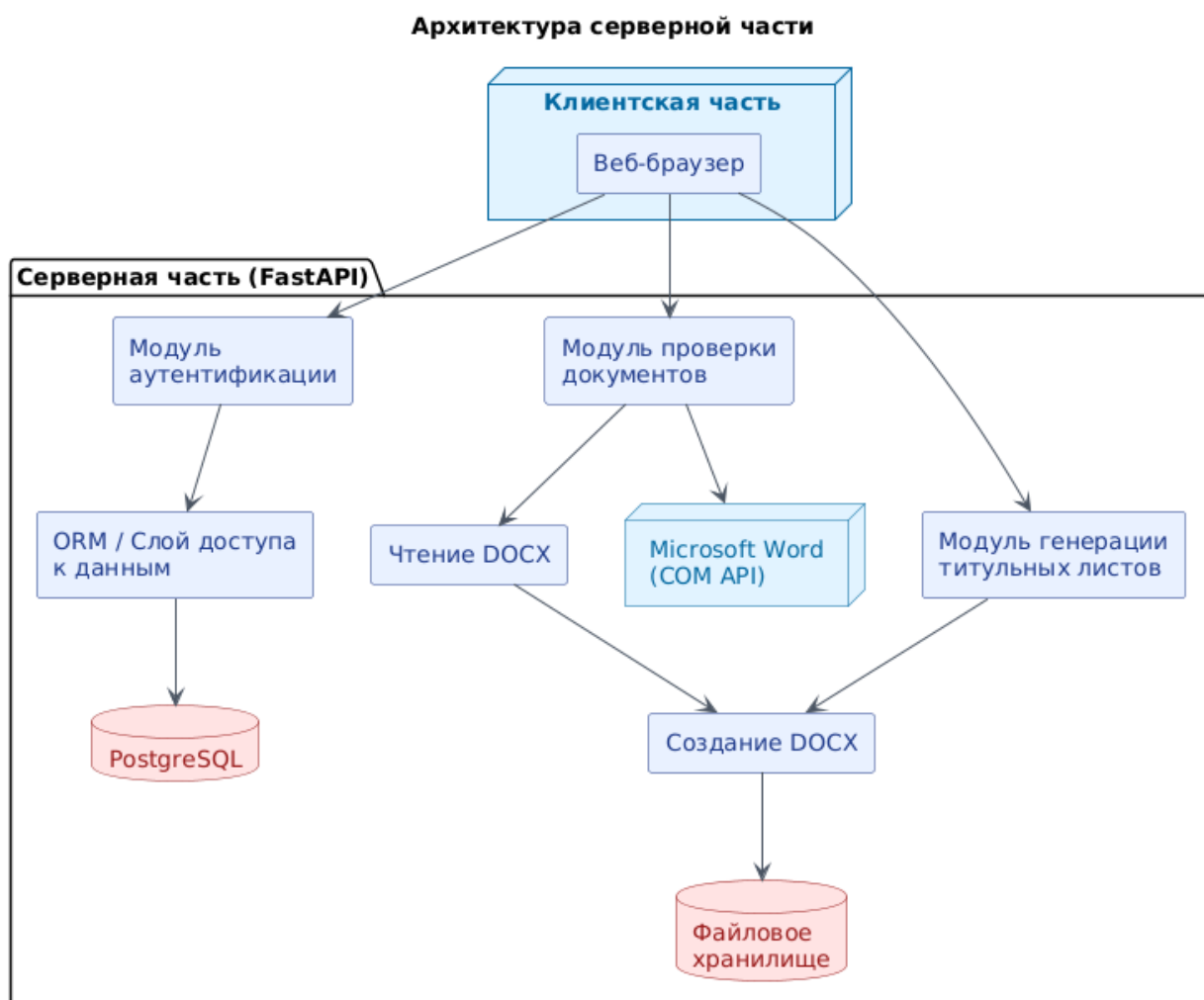


Рисунок 3.1 – Общая схема архитектуры системы

3.2 Модуль обработки учебно-методических документов

Модуль обработки учебно-методических документов является центральным компонентом системы, отвечающим за приём, анализ и проверку файлов формата DOCX. Он одинаково работает для методических указаний, учебных пособий и монографий, поскольку алгоритм проверки оформления и механизмы работы с документами универсальны. Различия между типами документов проявляются только на этапе генерации титульных листов, где используются разные наборы полей и шаблоны оформления.

3.2.1 Общая архитектура модуля

Модуль построен по многоуровневой архитектуре и включает следующие компоненты:

- **слой маршрутизации** — приём HTTP-запросов, валидация входных данных, маршрутизация к соответствующим обработчикам;
- **сервисный слой** — реализация бизнес-логики: проверка документов, генерация титульных листов, расчёт статистики;
- **слой доступа к данным** — объектно-реляционное отображение для взаимодействия с PostgreSQL;
- **слой утилит** — вспомогательные функции для работы с файлами, хеширования, логирования.

Такое разделение обеспечивает слабую связанность компонентов и упрощает тестирование и дальнейшее расширение.

3.2.2 Интеграция с Microsoft Word через COM-интерфейс

Ключевой особенностью модуля является использование COM-интерфейса Microsoft Word через библиотеку pywin32. При запуске проверки создаётся экземпляр приложения Word в фоновом режиме с отключёнными всплывающими окнами и диалогами. Это позволяет выполнять анализ и модификацию документов без видимого запуска графического интерфейса Word.

Основные операции, выполняемые через СОМ-интерфейс:

- открытие документа (`Documents.Open`) с возможностью только чтения или с правом записи;
- получение общей информации о документе: количество страниц, разделов, абзацев;
- обход всех абзацев документа и извлечение параметров форматирования;
- доступ к структуре страниц (поля, колонтитулы, нумерация);
- модификация документа: изменение цвета шрифта для выделения ошибок, добавление комментариев;
- сохранение изменённой копии и закрытие документа.

3.2.3 Алгоритм автоматической проверки оформления

Процесс проверки запускается после получения файла и его временно-го сохранения. Алгоритм реализован в классе `WordMethodicalChecker` и включает следующие этапы.

Этап 1. Предварительная обработка.

Файл сохраняется во временную директорию, создаётся уникальный идентификатор задачи для отслеживания прогресса. Если требуется сохранение исправленной версии (параметр `mark_document = True`), создаётся копия документа с суффиксом `_checked`, с которой будет работать модуль. Исходный файл остаётся неизменным.

Этап 2. Определение объёма работы.

Через СОМ-интерфейс открывается документ, вычисляется общее количество страниц с помощью метода `ComputeStatistics(WD_STATISTIC_PAGES)`. Эта информация используется для последующей группировки ошибок по страницам.

Этап 3. Фильтрация служебных элементов.

При обходе абзацев пропускаются:

- пустые абзацы — определяются по длине текста (менее 3 символов);

– абзацы, находящиеся внутри таблиц — метод `p.Range.Information(WD_WITHIN_TABLE)` возвращает `True` для таких элементов;

– заголовки — определяются по имени стиля: если в названии стиля присутствует подстрока "heading" или "заголов", абзац считается заголовком и пропускается.

Отфильтрованные элементы не участвуют в сравнении параметров оформления, так как к ним предъявляются иные требования или они не являются частью основного текста.

Этап 4. Извлечение параметров форматирования.

Для каждого значимого абзаца извлекаются следующие характеристики:

- **шрифт** — `p.Range.Font.Name`, эталонное значение "Times New Roman";
- **размер шрифта** — `p.Range.Font.Size`, эталонное значение 16 pt;
- **отступ первой строки** — `p.Range.ParagraphFormat.FirstLineIndent`, сначала проверяется формат самого абзаца, если значение близко к нулю, выполняется попытка получить отступ из стиля абзаца;
- **межстрочный интервал** — `p.Range.ParagraphFormat.LineSpacingRule`, эталонное значение `WD_LINE_SPACE_SINGLE` (одинарный интервал);
- **выравнивание текста** — анализируется при проверке номеров страниц.

Особое внимание уделяется случаю, когда в одном абзаце встречается неоднородное форматирование (например, разные размеры шрифта в одной строке). В таких ситуациях выполняется дополнительный обход всех слов внутри абзаца через коллекцию `p.Range.Words`. Для каждого слова извлекается его размер шрифта `w.Font.Size`. Если размер отличается от эталонного значения 16 pt более чем на 0,2 pt, и при этом слово не является знаком препинания (точка, запятая, скобка и т.п.), фиксируется ошибка «неверный размер шрифта» с указанием конкретного слова и номера страницы. Значения размера, равные 9999999,0, интерпретируются как ошибочные (такое значение возникает при попытке прочитать размер шрифта у объекта, не имеющего единого форматирования) и также приводят к фиксации нарушения. Такой

подход позволяет выявить ситуации, когда большая часть абзаца отформатирована правильно, но отдельные слова имеют неверный кегль, что часто встречается при копировании текста из внешних источников.

Этап 5. Сравнение с эталонами и фиксация ошибок.

Полученные параметры преобразуются в сантиметры (для отступов) и сравниваются с эталонными значениями:

- для отступа используется допуск 0,05 см: $\text{abs}(\text{indent_cm} - 1.25) > 0.05$;
- для размера шрифта используется допуск 0,2 pt: $\text{abs}(\text{size} - 16) > 0.2$;
- для шрифта и интервала выполняется точное сравнение (без допуска).

При обнаружении отклонения фиксируется номер страницы, тип нарушения и поясняющее сообщение. Номер страницы получается методом `p.Range.Information(WD_ACTIVE_END_PAGE_NUMBER)`. Каждая ошибка сохраняется в виде объекта `CheckIssue`, содержащего:

- `rule` — раздел проверки (например, "3" — для параметров текста, "4" — для нумерации);
- `severity` — степень важности ("ERROR" или "WARNING");
- `location` — местоположение в документе (страница или диапазон страниц);
- `message` — текст сообщения с описанием нарушения и рекомендацией по исправлению;
- `priority` — приоритет (используется для сортировки).

Этап 6. Проверка полей страницы.

После обработки всех абзацев выполняется проверка параметров страниц. Для каждого раздела документа (коллекция `doc.Sections`) извлекаются и проверяются настройки `PageSetup`:

- верхнее поле (`TopMargin`) — эталон 3,0 см;
- нижнее поле (`BottomMargin`) — эталон 2,25 см;
- левое поле (`LeftMargin`) — эталон 2,2 см;
- правое поле (`RightMargin`) — эталон 2,3 см.

Все поля проверяются в сантиметрах с допуском 0,05 см. При нарушении фиксируется номер раздела, и ошибка добавляется в отчёт.

Этап 7. Проверка нумерации страниц.

Проверка выполняется по всем разделам документа. Для каждого раздела анализируются верхний (Headers(WD_HEADER_FOOTER_PRIMARY)) и нижний (Footers(WD_HEADER_FOOTER_PRIMARY)) колонтитулы:

- Если в верхнем колонтитуле обнаружены поля с типом WD_FIELD_PAGE — нумерация присутствует в верхней части страницы, дополнительно проверяется выравнивание номера (должно быть по центру, WD_ALIGN_CENTER).
- Если номер найден только в нижнем колонтитуле — фиксируется ошибка о неверном расположении номера страницы.
- Если номер не найден ни в верхнем, ни в нижнем колонтитуле — фиксируется ошибка об отсутствии нумерации страниц.

Этап 8. Группировка ошибок.

Для повышения удобочитаемости отчёта последовательные страницы с одинаковыми типами ошибок объединяются в диапазоны. Например, если ошибки шрифта обнаружены на страницах 5, 6, 7, 8, отчёт покажет «Стр. 5–8» вместо четырёх отдельных записей. Группировка выполняется методом `_group_pages`, который сортирует номера страниц, находит в них непрерывные последовательности и формирует соответствующие записи. Для каждого диапазона создаётся одна запись в отчёте, а для всех страниц диапазона выполняется визуальное выделение ошибок.

Этап 9. Завершение проверки и освобождение ресурсов.

По окончании обхода всех абзацев и проверки полей и нумерации страниц выполняется сохранение изменений в документе (если `mark_document = True`) и закрытие документа через `doc.Close(False)`. После этого экземпляр Word завершается вызовом `self.word.Quit(False)`. Все временные файлы, созданные в процессе проверки (временная копия документа, файлы для отчёта), удаляются. Если на любом из этапов произошла ошибка, Word принудительно за-

вершается, а временные файлы удаляются в блоке `finally`, что гарантирует отсутствие «висящих» процессов и освобождение дискового пространства.

3.2.4 Формирование исправленной копии документа

Если в документе обнаружены ошибки и установлен флаг `mark_document` (активируется по умолчанию), модуль создаёт копию, в которой проблемные фрагменты визуально выделены. Алгоритм выделения работает следующим образом:

1. Для каждой группы ошибок определяется диапазон страниц, на которых они обнаружены.
2. Для первой страницы диапазона вычисляется начальная позиция через `doc.GoTo(1, 1, page)`. Константа 1 обозначает переход к странице, второй параметр — тип объекта (страница).
3. Для последней страницы диапазона вычисляется конечная позиция через `doc.GoTo(1, 1, page + 1)`. Если страница последняя в документе, конечная позиция определяется как `doc.Content.End`.
4. Область выделения задаётся через `doc.Range(start, end)`.
5. Цвет шрифта в выделенной области устанавливается в красный (`WD_COLOR_RED`) с помощью свойства `rng.Font.Color`.
6. К выделенной области добавляется комментарий с поясняющим сообщением через `doc.Comments.Add(rng, message)`.

Такой подход позволяет сохранить исходную структуру документа и одновременно наглядно показать автору все места, требующие исправления. Комментарии Word видны при наведении курсора и не нарушают вёрстку документа.

3.2.5 Генерация PDF-отчёта

Параллельно с модификацией документа формируется PDF-отчёт с использованием библиотеки `reportlab`. Процесс создания отчёта включает следующие шаги:

1. Регистрация шрифтов семейства Times New Roman (обычный, жирный, курсив, жирный курсив) для корректного отображения кириллицы. Библиотека reportlab не поддерживает кириллицу по умолчанию, поэтому требуется явная регистрация шрифтов с указанием пути к файлам .ttf.

2. Настройка параметров страницы: формат A4, поля 15 мм со всех сторон.

3. Формирование заголовочной части: название документа («Отчёт по проверке»), имя проверенного файла, дата и время проверки.

4. Определение итогового статуса:

- если есть ошибки уровня ERROR — статус «документ содержит критические ошибки» (отображается красным цветом);

- если есть только предупреждения (WARNING) — «документ содержит предупреждения» (оранжевый);

- если ошибок нет — «документ соответствует требованиям» (зелёный).

5. Построение таблицы ошибок с колонками: номер ошибки, раздел проверки, тип (ошибка/предупреждение), местоположение в документе, описание проблемы.

6. Стилизация таблицы: строки с ошибками выделяются красным фоном (BackgroundColor = #FEE2E2), строки с предупреждениями — жёлтым (BackgroundColor = #FEF3C7).

Отчёт сохраняется в директорию reports/ с именем, содержащим уникальный идентификатор, чтобы избежать конфликтов при одновременной работе нескольких пользователей.

3.2.6 Генерация титульных листов

Генерация титульных листов реализована через библиотеку python-docx без использования Microsoft Word. Это позволяет выполнять операцию быстро и без зависимости от установленного офисного пакета. В зависимости от типа документа используются разные наборы полей и шаблоны:

1. Для методических указаний: название, дисциплина, направление подготовки, составитель, рецензенты (с возможностью добавления нескольких), УДК, описание.

2. Для учебных пособий: автор, название, рецензенты, направления подготовки, значение А, ISBN, УДК, ББК, описание.

3. Для монографии: авторы (с возможностью добавления нескольких), название, город, год, УДК, ББК, ISBN, описание.

Алгоритм генерации единый для всех типов:

1. Создаётся новый пустой документ DOCX.
2. Устанавливаются поля страницы: верхнее 3 см, нижнее 2,25 см, левое 2,2 см, правое 2,3 см.
3. Задаётся стиль Normal: шрифт Times New Roman, размер 16 pt.
4. Последовательно добавляются абзацы с заданным выравниванием (центр, право, лево) и форматированием.
5. Для каждого абзаца через вспомогательную функцию `add_paragraph` устанавливаются параметры: текст, выравнивание, размер шрифта, жирное начертание, отступы до и после абзаца, межстрочный интервал.
6. После заполнения первой страницы добавляется разрыв раздела (`doc.add_section(WD_SECTION_START.NEW_PAGE)`), и формируется вторая страница.
7. Вторая страница содержит библиографическое описание, аннотацию и выходные данные — в зависимости от типа документа набор полей различается.
8. Готовый документ сохраняется во временную директорию, после чего отправляется пользователю и удаляется с сервера.

3.2.7 Управление сессиями и обновление токенов

Для поддержания авторизации пользователя без многократного ввода пароля используется двухуровневая система токенов: `access`-токен (срок действия 60 минут) и `refresh`-токен (срок действия 30 дней). `Access`-токен подписывается секретным ключом и содержит идентификатор пользователя и его

email. Refresh-токен генерируется как случайная строка, его хеш сохраняется в базе данных вместе с метаданными (IP-адрес, User-Agent).

При получении запроса к защищённому ресурсу сервер проверяет access-токен: если он истёк, клиент получает ошибку 401 Unauthorized и автоматически отправляет refresh-токен на эндпоинт /auth/refresh. Сервер проверяет, что refresh-токен не отозван (поле revoked), не истёк и принадлежит тому же пользователю, после чего выдаёт новый access-токен. Refresh-токен при этом остаётся без изменений, что позволяет обновлять access-токен многократно без повторного входа пользователя.

При выходе из системы соответствующий refresh-токен отмечается в базе данных как отозванный (revoked = True), что делает его непригодным для дальнейшего использования. Это позволяет администратору в случае подозрения на компрометацию учётной записи отозвать все токены пользователя одной командой.

3.2.8 Взаимодействие с базой данных через ORM

Для взаимодействия с PostgreSQL используется объектно-реляционное отображение SQLAlchemy. ORM позволяет работать с записями базы данных как с обычными объектами Python, скрывая детали формирования SQL-запросов. Каждая таблица описывается отдельным классом, наследующим от Base, с указанием типов полей, ограничений и связей.

Пример описания таблицы manual:

```
class Manual(Base):
    __tablename__ = "manual"
    id_manual = Column(Integer, primary_key=True, index=True)
    manual_name = Column(String(255), nullable=False)
    fio_user = Column(String(255), nullable=False)
    faculty_code = Column(String(10), nullable=False)
    department_name = Column(String(255), nullable=False)
    file_hash = Column(String(64), unique=True, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```

Сессии SQLAlchemy управляются через контекстный менеджер: при каждом запросе создаётся новая сессия, которая автоматически закрывается после окончания обработки. Это гарантирует, что соединения с базой данных не «утекают» и корректно возвращаются в пул.

Все запросы к базе данных используют параметризацию (через ORM-методы `filter` и `filter_by`), что полностью исключает возможность SQL-инъекций. При сохранении документа с успешно пройденной проверкой хеш файла проверяется на уникальность; если запись с таким хешем уже существует, операция вставки не выполняется, и пользователь получает соответствующее уведомление.

3.2.9 Предотвращение дублирования

Для исключения повторной загрузки одинаковых документов используется вычисление хеш-суммы файла (алгоритм SHA-256). Хеш вычисляется блоками по 8192 байта, что позволяет обрабатывать файлы любого размера без полной загрузки в память.

При успешной проверке (документ не содержит ошибок) хеш сохраняется в базе данных вместе с метаданной: название документа, ФИО автора, код факультета, название кафедры, дата загрузки. При каждой новой загрузке система выполняет следующие действия:

1. Вычисляется хеш загружаемого файла.
2. Выполняется запрос к базе данных на поиск записи с таким же хешем.
3. Если запись найдена, загрузка отклоняется, пользователю возвращается сообщение «Данный документ уже был загружен ранее».
4. Если запись не найдена, документ сохраняется, и в базу добавляется новая запись.

Это предотвращает накопление дубликатов и экономит дисковое пространство.

3.2.10 Обработка ошибок и восстановление

При работе с COM-интерфейсом Microsoft Word возможны различные сбои: документ может быть повреждён, Word может зависнуть или завершиться аварийно. В модуле реализована многоуровневая обработка исключений:

1. На уровне COM-вызовов — каждый вызов метода Word обернут в блок try-except. При возникновении ошибки (например, попытка открыть повреждённый файл) выполняется корректное закрытие документа и завершение процесса Word через `self.word.Quit(False)`. Пользователь получает сообщение с указанием причины ошибки и предложением проверить файл.

2. На уровне проверки документа — при сбое на любом этапе (фильтрация абзацев, извлечение параметров, группировка ошибок) проверка прерывается, пользователю возвращается код ошибки 500 Internal Server Error с пояснением. Временные файлы удаляются в блоке finally, чтобы не засорять дисковое пространство.

3. На уровне базы данных — при нарушении уникальности (попытка вставить дубликат) или других ограничениях целостности формируется понятное сообщение об ошибке без раскрытия деталей SQL-запроса.

Дополнительно реализован механизм тайм-аутов: если проверка документа занимает более 600 секунд (10 минут), операция прерывается, и пользователь получает сообщение о необходимости уменьшить размер файла или разбить его на части.

3.2.11 Хранение данных

Для хранения метаданных используется таблица `manual` в PostgreSQL, содержащая следующие поля:

- `id_manual` — первичный ключ (автоинкремент);
- `manual_name` — название документа;
- `fio_user` — ФИО автора;
- `faculty_code` — код факультета;

- department_name — название кафедры;
- file_hash — уникальный хеш (SHA-256);
- created_at — дата и время загрузки.

Сами файлы хранятся в файловой системе в директориях checked_files/ (проверенные документы с выделенными ошибками) и reports/ (PDF-отчёты). В базе хранятся только пути к этим файлам. Такой подход снижает нагрузку на СУБД и позволяет хранить файлы любого размера без ограничений.

3.2.12 Логирование

Все операции модуля логируются с использованием встроенного модуля logging. Настроены два уровня логирования:

1. INFO — фиксируются штатные операции: загрузка файла, успешное завершение проверки, генерация титульного листа, отправка результата пользователю, авторизация и выход пользователя.

2. ERROR — фиксируются сбои: невозможность открыть документ через COM-интерфейс, ошибка при сохранении файла, проблема с подключением к базе данных, тайм-аут проверки.

Логи сохраняются в файл с ротацией по дате, что позволяет администратору системы анализировать причины отказов, отслеживать подозрительную активность и своевременно устранять неполадки.

3.2.13 Заключение

Разработанный модуль обработки учебно-методических документов реализует полный цикл работы с документами формата DOCX: приём файлов, автоматическую проверку оформления через COM-интерфейс Microsoft Word, группировку ошибок, формирование исправленной копии с визуальной разметкой, генерацию PDF-отчётов, создание титульных листов через python-docx (с различными шаблонами для методических указаний, учебных пособий и монографий), управление сессиями через двухуровневую систему токенов, предотвращение дублирования с помощью хеширования, надёжное хранение метаданных в PostgreSQL, а также многоуровневую обработку

ошибок и логирование. Все компоненты модуля спроектированы с учётом требований к производительности, расширяемости и отказоустойчивости.

3.3 База данных и её проектирование

В качестве системы управления базами данных выбрана PostgreSQL. Эта СУБД обеспечивает высокую надёжность хранения информации, полную поддержку транзакций, соблюдение принципов ACID, а также располагает встроенными механизмами для контроля целостности данных на уровне ограничений и ссылочной целостности.

Подключение к базе данных реализовано через отдельный модуль `database.py`, который отвечает за конфигурацию параметров соединения, инициализацию подключений и их корректное завершение. В приложении задействован пул соединений — это позволяет переиспользовать уже установленные подключения, снижая накладные расходы на установку нового соединения для каждого запроса, и эффективно обслуживать большое количество одновременно работающих пользователей.

Для описания структур данных применяется объектно-реляционное отображение (ORM) SQLAlchemy. Каждая сущность предметной области представлена в коде отдельным классом, который описывает схему соответствующей таблицы: перечень полей, их типы, ограничения, а также связи с другими таблицами (один-ко-многим, многие-ко-многим). Такой подход упрощает взаимодействие с базой данных и снижает вероятность ошибок при формировании SQL-запросов.

3.3.1 ER-модель данных

На основе анализа предметной области были выделены основные сущности системы и связи между ними. ER-диаграмма отражает структуру данных и взаимосвязи между пользователями, кафедрами, факультетами и загруженными документами.

ER-диаграмма базы данных представлена на рисунке 3.2.

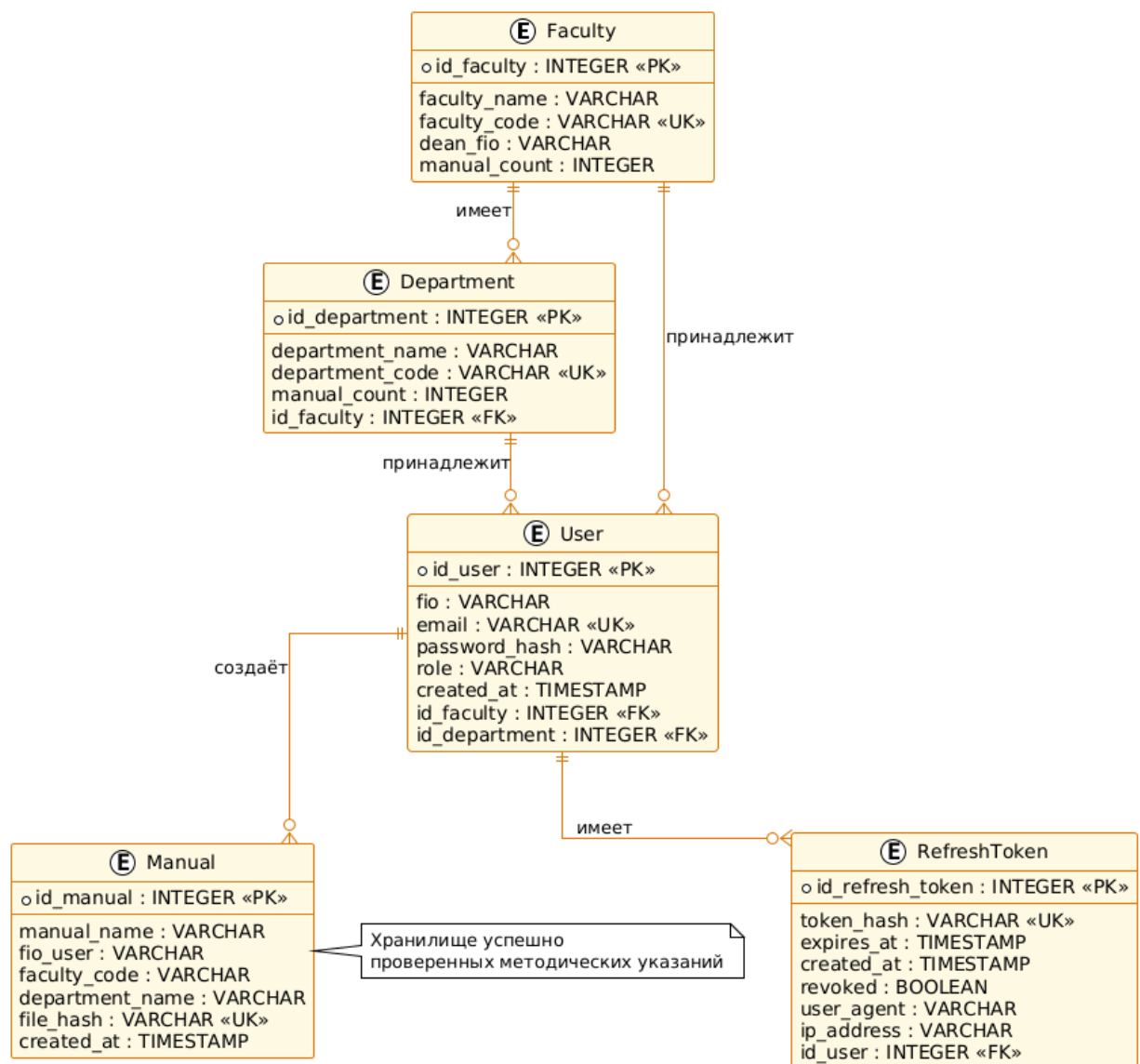


Рисунок 3.2 – ER-диаграмма базы данных

В модели центральное место занимает сущность «manual», связанная с пользователями и организационной структурой (кафедры и факультеты). Также выделена сущность токенов обновления, обеспечивающая механизм аутентификации пользователей.

К числу основных сущностей базы данных относятся:

- пользователи (users) — содержат учётные записи лиц, работающих с системой (авторы, преподаватели, сотрудники);
- факультеты (faculty) — подразделения университета, к которым привязаны пользователи и кафедры;
- кафедры (department) — структурные единицы внутри факультетов;

- документы (manual) — таблица, хранящая сведения о всех типах учебно-методических материалов (методические указания, учебные пособия, монографии), успешно прошедших проверку;
- токены обновления (refresh_tokens) — используются для поддержания сеансов авторизации.

3.3.2 Нормализация

Структура базы данных спроектирована в соответствии с требованиями третьей нормальной формы.

Каждая таблица оснащена первичным ключом, все поля содержат неделимые значения, а взаимосвязи между сущностями организованы через механизм внешних ключей. Избыточность данных устранена путём декомпозиции информации на логически независимые таблицы (факультеты, кафедры, пользователи).

Помимо этого, в схеме предусмотрены дополнительные ограничения, гарантирующие целостность данных. К ним относятся уникальные индексы (например, для адреса электронной почты пользователя и хеш-суммы файла документа), а также проверочные условия (например, допустимые значения для роли пользователя).

3.3.3 Реляционная модель данных

На основе ER-модели построена реляционная модель базы данных, отражающая структуру таблиц и их взаимосвязи.

Схема реляционной модели представлена на рисунке 3.3.

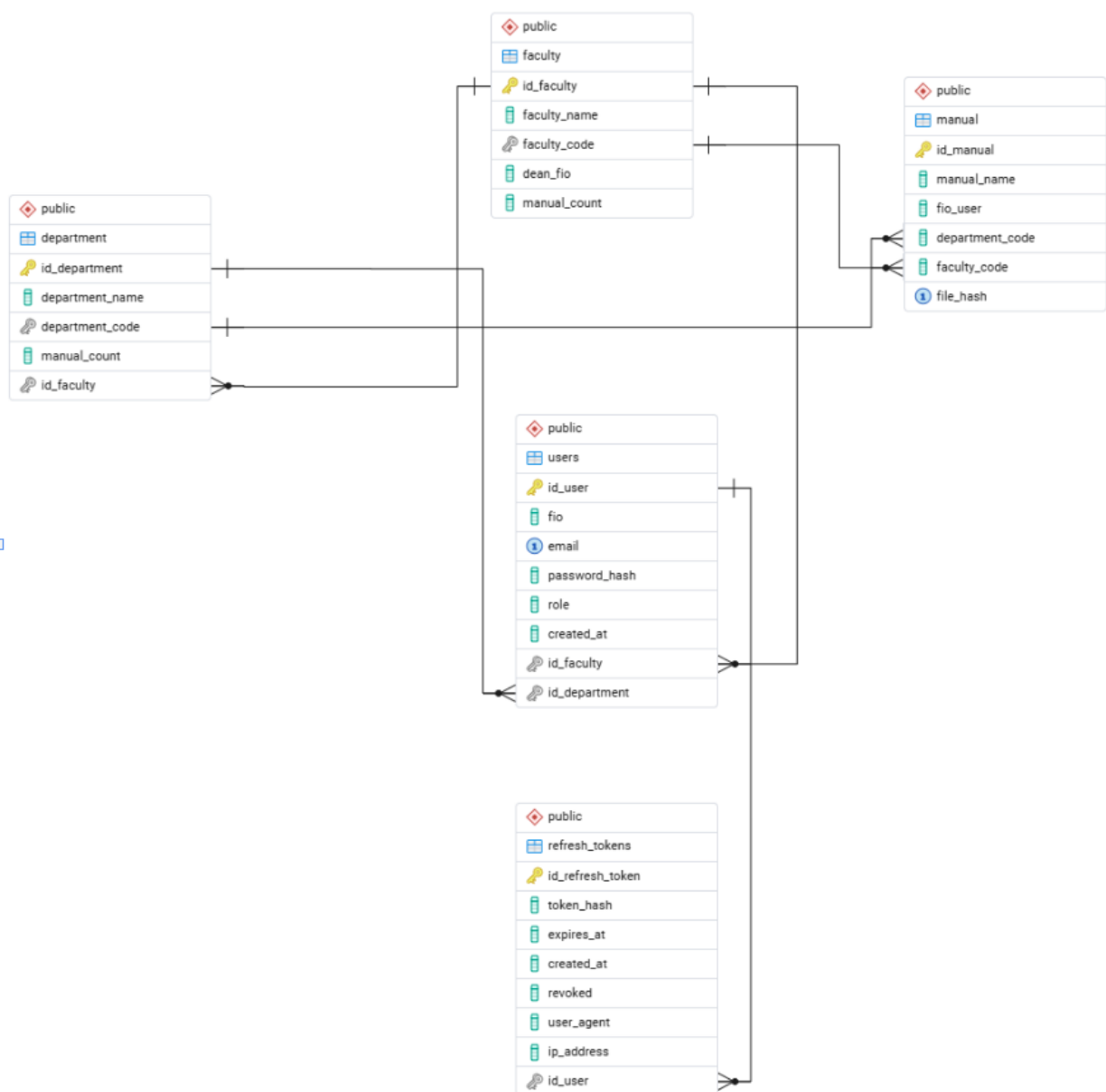


Рисунок 3.3 – Реляционная модель базы данных

Реляционная модель включает таблицы пользователей, факультетов, кафедр, документов (методических указаний, учебных пособий, монографий) и токенов обновления. Связи между таблицами реализованы через внешние ключи, что обеспечивает целостность данных и согласованность структуры.

Особое значение имеет таблица «manual», используемая для хранения информации о всех загружаемых документах и предотвращения их дублирования.

3.3.4 Описание структуры базы данных

Центральное место в базе данных занимает таблица «manual», предназначенная для хранения сведений о загружаемых документах независимо от их типа. Для каждой записи фиксируются следующие данные: название документа, фамилия, имя, отчество автора, код факультета, название кафедры, а также уникальная хеш-сумма файла. Хеш вычисляется на основе содержимого документа и служит для предотвращения дублирования — повторная загрузка одного и того же файла не приведёт к созданию лишней записи в базе.

Связь пользователей с организационной структурой обеспечивается через таблицы факультетов и кафедр. Каждый пользователь закреплён за конкретной кафедрой и факультетом, что позволяет отслеживать принадлежность авторов документов к соответствующим подразделениям и при необходимости формировать статистику по кафедрам или факультетам.

Файлы документов размещаются непосредственно в файловой системе сервера — это упрощает работу с большими объёмами данных и обеспечивает быстрый доступ к содержимому. В базе данных при этом сохраняются только метаданные (название, автор, дата загрузки) и уникальные идентификаторы, необходимые для поиска файла в файловом хранилище. Такое разделение позволяет снизить нагрузку на СУБД и эффективно масштабировать систему по мере увеличения количества загружаемых документов.

В таблицах 3.1–3.5 приведено подробное описание структуры всех таблиц базы данных.

Таблица 3.1 – Атрибуты таблицы «faculty» (Факультеты)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_faculty	SERIAL	
	*	faculty_name	VARCHAR	255
UK	*	faculty_code	INTEGER	
	*	dean_fio	VARCHAR	255

Продолжение таблицы 3.1

Key Type	Optionality	Column name	Data Type	Size
		manual_count	INTEGER	

Таблица 3.2 – Атрибуты таблицы «department» (Кафедры)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_department	SERIAL	
	*	department_name	VARCHAR	255
UK	*	department_code	INTEGER	
		manual_count	INTEGER	
FK	*	id_faculty	INTEGER	

Таблица 3.3 – Атрибуты таблицы «users» (Пользователи)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_user	SERIAL	
	*	fio	VARCHAR	255
UK	*	email	VARCHAR	255
	*	password_hash	VARCHAR	255
	*	role	VARCHAR	50
		created_at	TIMESTAMP	
FK	*	id_faculty	INTEGER	
FK	*	id_department	INTEGER	

Таблица 3.4 – Атрибуты таблицы «manual» (Документы)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_manual	SERIAL	
	*	manual_name	VARCHAR	255
	*	fio_user	VARCHAR	255

Продолжение таблицы 3.4

Key Type	Optionality	Column name	Data Type	Size
	*	department_code	INTEGER	
	*	faculty_code	INTEGER	
UK	*	file_hash	VARCHAR	64

Таблица 3.5 – Атрибуты таблицы «refresh_tokens» (Токены обновления)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_refresh_token	SERIAL	
UK	*	token_hash	VARCHAR	255
	*	expires_at	TIMESTAMP	
		created_at	TIMESTAMP	
	*	revoked	BOOLEAN	
		user_agent	VARCHAR	500
		ip_address	VARCHAR	100
FK	*	id_user	INTEGER	

4 Рабочий проект

4.1 Маршруты приложения

В таблице 4.1 приведены основные маршруты (endpoints) REST API серверной части, реализующие ключевую функциональность информационно-аналитической системы: автоматическую проверку документов, генерацию титульных листов и отправку файлов в редакционно-издательский отдел.

Таблица 4.1 – Маршруты приложения

URI	HTTP-метод	Назначение
Модуль проверки документов (/checker)		
/checker/check	POST	Загрузка и автоматическая проверка документа DOCX
/checker/my/{doc_type}	GET	Получение списка успешно проверенных документов пользователя
/checker/download-checked/{filename}	GET	Скачивание документа с визуально выделенными ошибками
/checker/report/{filename}	GET	Скачивание PDF-отчёта с результатами проверки
/checker/ping	GET	Проверка работоспособности модуля
Модуль генерации титульных листов (/title-page)		
/title-page/generate	POST	Генерация титульного листа для методических указаний
/title-page/generate-tutorial	POST	Генерация титульного листа для учебного пособия
/title-page/generate-monograph	POST	Генерация титульного листа для монографии
Модуль отправки в РИО (/rio)		
/rio/upload-step1	POST	Шаг 1: загрузка двух вспомогательных файлов

URI	HTTP-метод	Назначение
/rio/upload-step2	POST	Шаг 2: загрузка и проверка основного документа
/rio/submit-step3	POST	Шаг 3: отправка всех трёх файлов на email РИО
/rio/reset	POST	Сброс сессии отправки (очистка временных файлов)

4.2 Спецификация контроллеров приложения

Ниже приведено описание ключевых методов контроллеров, реализующих автоматическую проверку документов, генерацию титульных листов и отправку файлов в РИО. Эти компоненты составляют основную функциональность информационно-аналитической системы.

4.2.1 Контроллер модуля проверки документов

В таблице 4.2 приведено описание методов класса CheckerController, который реализует алгоритм автоматической проверки оформления документов через COM-интерфейс Microsoft Word.

Таблица 4.2 – Спецификация методов класса CheckerController

Название	Доступ	Описание
check	public	Загрузка DOCX-файла, запуск проверки (через COM-интерфейс Word). Возвращает JSON с результатами, ссылку на исправленный документ и PDF-отчёт.
getMyDocuments	public	Возврат списка ранее проверенных документов пользователя с их метаданными (название, дата, факультет, кафедра).

Название	Доступ	Описание
downloadCheckedFile	public	Скачивание DOCX с визуально выделенными ошибками (красный цвет, комментарии).
openPdfReport	public	Скачивание PDF-отчёта со списком ошибок и рекомендациями.
ping	public	Проверка работоспособности модуля (используется для диагностики).

4.2.2 Контроллер модуля генерации титульных листов

В таблице 4.3 приведено описание методов класса TitlePageController, который создаёт титульные листы для разных типов документов с использованием библиотеки python-docx.

Таблица 4.3 – Спецификация методов класса TitlePageController

Название	Доступ	Описание
generateManualTitle	public	Генерация DOCX-файла титульного листа для методических указаний по шаблону.
generateTutorialTitle	public	Генерация DOCX-файла титульного листа для учебного пособия.
generateMonographTitle	public	Генерация DOCX-файла титульного листа для монографии.

4.2.3 Контроллер модуля отправки в РИО

В таблице 4.4 приведено описание методов класса RioController, который реализует трёхшаговую отправку файлов по электронной почте через SMTP.

Таблица 4.4 – Спецификация методов класса RioController

Название	Доступ	Описание
uploadStep1	public	Приём двух вспомогательных файлов (произвольные документы), временное сохранение в директории пользователя.
uploadStep2	public	Приём основного документа, запуск проверки (через СОМ-интерфейс). При ошибках блокирует переход к шагу 3.
submitStep3	public	Формирование email с вложениями (три файла), отправка через SMTP-сервер, очистка временных файлов.
resetSession	public	Принудительный сброс сессии, удаление всех временных файлов пользователя.

4.3 Системное тестирование разработанного web-сайта

В процессе разработки было проведено функциональное тестирование ключевых компонентов бэкенда: проверки документов, генерации титульных листов и отправки в РИО. Тестирование выполнялось с использованием инструментов (FastAPI, Postman) и проверки выходных данных (DOCX, PDF, email).

4.3.1 Тестирование модуля проверки документов

Сценарий 1: Проверка документа, соответствующего требованиям.

- **Входные данные:** DOCX-файл методических указаний, оформленный по ГОСТ (шрифт Times New Roman, размер 16 pt, отступы 1,25 см, одинарный интервал).
- **Действие:** Отправка POST-запроса на /checker/check.
- **Ожидаемый результат:** Система через СОМ-интерфейс Word выполняет проверку, не находит ошибок. В ответе has_errors: false, документ автоматически сохраняется в БД. PDF-отчёт не формируется.

Сценарий 2: Проверка документа с ошибками оформления.

- **Входные данные:** DOCX-файл с намеренными нарушениями (шрифт Arial, размер 12 pt, отсутствие отступа).
- **Действие:** Отправка POST-запроса на /checker/check.
- **Ожидаемый результат:** Алгоритм проверки выявляет ошибки, фиксирует номера страниц и группирует их в диапазоны. В ответе has_errors: true, формируются:
 - исправленная копия DOCX с выделением ошибок красным цветом и комментариями;
 - PDF-отчёт с таблицей ошибок (номер, тип, местоположение, описание);
 - ссылки на оба файла возвращаются клиенту.

4.3.2 Тестирование модуля генерации титульных листов

Сценарий 1: Генерация титульного листа для методических указаний.

- **Входные данные:** JSON с полями: manual_title, discipline_name, direction_code, compiler_name, reviewers (массив), udk, description.
- **Действие:** Отправка POST-запроса на /title-page/generate.
- **Ожидаемый результат:** Библиотека python-docx формирует DOCX-файл с двумя страницами, соответствующими шаблону (шапка, блок утверждения, название, дисциплина, выходные данные, УДК, рецензенты, аннотация). Файл возвращается клиенту.

Сценарий 2: Генерация титульного листа для монографии.

- **Входные данные:** JSON с полями: authors (массив), monograph_title, city, year, udk, bbk, isbn, description.
- **Действие:** Отправка POST-запроса на /title-page/generate-monograph.
- **Ожидаемый результат:** Формируется DOCX-файл, содержащий на первой странице авторов (в одну строку) и название монографии, на второй — УДК, ББК, ISBN, аннотацию и сведения об авторских правах.

4.3.3 Тестирование модуля отправки в РИО

Сценарий 1: Полный успешный цикл отправки.

- **Шаг 1 (upload-step1):** POST-запрос с двумя вспомогательными файлами (например, PDF и TXT). Система сохраняет их во временную директорию и возвращает next_step: 2.
- **Шаг 2 (upload-step2):** POST-запрос с основным DOCX-файлом. Система запускает проверку через COM-интерфейс. Если ошибок нет, возвращает next_step: 3.
- **Шаг 3 (submit-step3):** POST-запрос с JSON-полем comment (до 1000 символов). Система формирует email (тема: «Новые методические материалы от [ФИО]», тело: информация об отправителе и комментарий). Все три файла прикрепляются как вложения. Email отправляется через SMTP. Временные файлы удаляются.

Сценарий 2: Ошибка на шаге 2 (некорректный файл).

- **Входные данные:** DOCX-файл с ошибками оформления.
- **Действие:** POST-запрос на /rio/upload-step2.
- **Ожидаемый результат:** Модуль проверки выявляет ошибки, сервер возвращает HTTP-статус 422 и сообщение «В данном документе «имя_файла» имеются ошибки, выполните полную проверку файла». Переход к шагу 3 блокируется, сессия не изменяется.

4.3.4 Результаты тестирования

В ходе функционального тестирования было подтверждено, что:

- модуль проверки документов корректно анализирует параметры форматирования через COM-интерфейс Word, выявляет ошибки, группирует их по страницам и формирует исправленную копию с визуальным выделением;
- модуль генерации титульных листов создаёт DOCX-файлы для трёх типов документов в соответствии с утверждёнными шаблонами;

– модуль отправки в РИО обеспечивает надёжное временное хранение файлов и отправку email с вложениями через SMTP.

Все ключевые компоненты бэкенда функционируют в соответствии с требованиями технического задания. Система готова к промышленной эксплуатации в редакционно-издательском отделе вуза.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационно-аналитическая система автоматизации проверки учебно-методических материалов редакционно-издательского отдела. Разработанная система позволяет выполнять автоматизированную проверку документов на соответствие требованиям оформления, формировать отчёты о выявленных нарушениях, генерировать титульные листы и обеспечивать централизованное хранение данных.

В ходе работы был проведён анализ предметной области, связанной с организацией процессов проверки и сопровождения учебно-методической документации. Особое внимание было уделено проблемам ручной проверки документов, требованиям к оформлению учебных материалов и существующим подходам к автоматизации документооборота. Выполненный анализ позволил сформировать требования к функциональности разрабатываемой системы.

Разработанная система реализует следующие основные возможности:

- аутентификацию и авторизацию пользователей;
- загрузку и обработку учебно-методических документов;
- автоматическую проверку параметров форматирования;
- формирование PDF-отчётов по результатам проверки;
- генерацию титульных листов документов;
- взаимодействие пользователей с редакционно-издательским отделом через веб-интерфейс.

Для реализации системы использована клиент-серверная архитектура. Серверная часть разработана с использованием FastAPI и PostgreSQL, а клиентская часть реализована с применением HTML, CSS и JavaScript. Для хранения и обработки данных использована технология ORM SQLAlchemy.

В процессе разработки было проведено тестирование системы, включающее проверку корректности обработки документов, формирования отчётов, работы механизмов аутентификации и взаимодействия клиентской и сервер-

ной части. Результаты тестирования подтвердили корректность функционирования разработанной системы и соответствие реализованного функционала поставленным требованиям.

Основные результаты выпускной квалификационной работы:

1. Проведён анализ процессов проверки учебно-методических материалов и существующих средств автоматизации документооборота.
2. Разработана архитектура информационно-аналитической системы.
3. Реализована серверная часть системы с использованием FastAPI и PostgreSQL.
4. Разработана клиентская часть веб-приложения.
5. Реализованы модули проверки документов, генерации титульных листов и формирования PDF-отчётов.
6. Проведено тестирование системы и подтверждена корректность её работы.

Все задачи, поставленные в начале выпускной квалификационной работы, были успешно решены. Таким образом, цель выпускной квалификационной работы — разработка информационно-аналитической системы автоматизации проверки учебно-методических материалов редакционно-издательского отдела — была достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Питер, 2019. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.
2. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2017. – 768 с. – ISBN 978-5-97060-384-3. – Текст : непосредственный.
3. Python Documentation : сайт. – URL: <https://docs.python.org/3/> (дата обращения: 24.04.2026). – Текст : электронный.
4. PEP 8 – Style Guide for Python Code : сайт. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 24.04.2026). – Текст : электронный.
5. Параллельное программирование в Python (threading) : сайт. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 24.04.2026). – Текст : электронный.
6. Logging HOWTO : сайт. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 24.04.2026). – Текст : электронный.
7. Буэно, С. Разработка API с FastAPI / С. Буэно. – Москва : ДМК Пресс, 2022. – 384 с. – ISBN 978-5-97060-948-8. – Текст : непосредственный.
8. FastAPI Documentation : сайт. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 24.04.2026). – Текст : электронный.
9. Pydantic Documentation : сайт. – URL: <https://docs.pydantic.dev/> (дата обращения: 24.04.2026). – Текст : электронный.
10. Uvicorn Documentation : сайт. – URL: <https://www.uvicorn.org/> (дата обращения: 24.04.2026). – Текст : электронный.
11. Starlette Documentation : сайт. – URL: <https://www.starlette.io/> (дата обращения: 24.04.2026). – Текст : электронный.
12. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2005. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.

13. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2003. – 1440 с. – ISBN 978-5-8459-0527-7. – Текст : непосредственный.
14. Грофф, Дж. Р. SQL: полное руководство / Дж. Р. Грофф, П. Н. Вайнберг, Э. Дж. Оппель. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 960 с. – ISBN 978-5-907144-26-5. – Текст : непосредственный.
15. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2022. – 336 с. – ISBN 978-5-9775-3960-9. – Текст : непосредственный.
16. PostgreSQL Documentation : сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
17. SQLAlchemy Documentation : сайт. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.
18. Alembic – Database Migrations for SQLAlchemy : сайт. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.
19. Psycopg – PostgreSQL adapter for Python : сайт. – URL: <https://www.psycopg.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
20. python-docx Documentation : сайт. – URL: <https://python-docx.readthedocs.io/> (дата обращения: 24.04.2026). – Текст : электронный.
21. pywin32 Documentation : сайт. – URL: <https://github.com/mhammond/pywin32> (дата обращения: 24.04.2026). – Текст : электронный.
22. Стандарт открытого формата Office Open XML : сайт. – URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/> (дата обращения: 24.04.2026). – Текст : электронный.
23. JWT Introduction : сайт. – URL: <https://jwt.io/introduction> (дата обращения: 24.04.2026). – Текст : электронный.
24. OAuth 2.0 Authorization Framework : сайт. – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 24.04.2026). – Текст : электронный.

25. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2013. – 464 с. – ISBN 978-5-496-00487-9. – Текст : непосредственный.
26. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.
27. Рамбо, Дж. UML 2.0. Объектно-ориентированное моделирование и разработка / Дж. Рамбо, М. Блаха. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
28. Unified Modeling Language Specification, Version 2.5.1 : сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (дата обращения: 24.04.2026). – Текст : электронный.
29. draw.io Documentation : сайт. – URL: <https://www.drawio.com/doc/> (дата обращения: 24.04.2026). – Текст : электронный.
30. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3. – Текст : непосредственный.
31. Git Documentation : сайт. – URL: <https://git-scm.com/doc> (дата обращения: 24.04.2026). – Текст : электронный.
32. Дэкетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дэкетт. – Москва : Эксмо, 2014. – 480 с. – ISBN 978-5-699-64193-2. – Текст : непосредственный.
33. Дэкетт, Д. JavaScript и jQuery. Интерактивная web-разработка / Д. Дэкетт. – Москва : Эксмо, 2017. – 640 с. – ISBN 978-5-699-80285-2. – Текст : непосредственный.
34. Фрейн, Б. HTML5 и CSS3. Разработка современных web-сайтов / Б. Фрейн. – 2-е изд. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-4461-0526-2. – Текст : непосредственный.
35. HTML : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 24.04.2026). – Текст : электронный.

36. CSS : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 24.04.2026). – Текст : электронный.
37. JavaScript : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 24.04.2026). – Текст : электронный.
38. HTTP : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 24.04.2026). – Текст : электронный.
39. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – RFC 9110. – 2022. – URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (дата обращения: 24.04.2026). – Текст : электронный.
40. Филдинг, Р. Архитектурный стиль REST и проектирование сетевых приложений : сайт. – URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 24.04.2026). – Текст : электронный.
41. Оккен, Б. pytest. Быстрое тестирование приложений на Python / Б. Оккен. – Санкт-Петербург : Питер, 2020. – 320 с. – ISBN 978-5-4461-1265-4. – Текст : непосредственный.
42. PyCharm Documentation : сайт. – URL: <https://www.jetbrains.com/pycharm/documentation/> (дата обращения: 24.04.2026). – Текст : электронный.
43. Bootstrap Documentation : сайт. – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 24.04.2026). – Текст : электронный.
44. Docker Documentation : сайт. – URL: <https://docs.docker.com/> (дата обращения: 24.04.2026). – Текст : электронный.
45. NGINX Documentation : сайт. – URL: <https://nginx.org/en/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
46. ReportLab Documentation : сайт. – URL: <https://docs.reportlab.com/> (дата обращения: 24.04.2026). – Текст : Redirecting... git-scm.com электронный.
47. asyncio Documentation : сайт. – URL: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 24.04.2026). – Текст : электронный.
48. Кнут, Д. Э. Искусство программирования. Том 1. Основные алгоритмы / Д. Э. Кнут. – 4-е изд. – Москва : Вильямс, 2022. – 720 с. – ISBN 978-5-8459-2075-1. – Текст : непосредственный.

49. Лопес, Ф. SQLAlchemy. Архитектура и паттерны / Ф. Лопес. – Москва : ДМК Пресс, 2020. – 384 с. – ISBN 978-5-97060-765-0. – Текст : непосредственный.
50. Любанович, Б. Простой Python. Современный стиль программирования / Б. Любанович. – Санкт-Петербург : Питер, 2020. – 480 с. – ISBN 978-5-4461-1269-2. – Текст : непосредственный.
51. Вурс, Д. Python. Карманный справочник / Д. Вурс. – Москва : Эксмо, 2022. – 320 с. – ISBN 978-5-04-164234-5. – Текст : непосредственный.
52. Python Tutorial : сайт. – URL: <https://docs.python.org/3/tutorial/> (дата обращения: 25.04.2026). – Текст : электронный.
53. Рамальо, Л. Python. К вершинам мастерства. Том 2 / Л. Рамальо. – Москва : ДМК Пресс, 2023. – 672 с. – ISBN 978-5-97060-985-3. – Текст : непосредственный.
54. Дауни, А. Б. Think Python. Аналитический подход к программированию / А. Б. Дауни. – 3-е изд. – Санкт-Петербург : Питер, 2024. – 320 с. – ISBN 978-5-4461-2154-4. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Фрагменты исходного кода программы

main.py

```
1 import asyncio
2 import sys
3
4 from fastapi import FastAPI
5 from fastapi.staticfiles import StaticFiles
6 from fastapi.responses import FileResponse
7
8 from app.db.base import Base
9 from app.db.session import engine
10 from app.routers import admin, auth, checker, statistics, title_page, rio
11
12 if sys.platform == 'win32':
13     asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
14 app = FastAPI(title="Diplom Checker API")
15
16 Base.metadata.create_all(bind=engine)
17
18 app.mount("/static", StaticFiles(directory="app/static"), name="static")
19
20 app.include_router(auth.router)
21 app.include_router(checker.router)
22 app.include_router(admin.router)
23 app.include_router(statistics.router)
24 app.include_router(title_page.router)
25 app.include_router(rio.router)
26
27
28 @app.get("/")
29 def root():
30     return FileResponse("app/templates/index.html")
```

checker.py

```
1 import os
2 import shutil
3 from dataclasses import dataclass, field
4 from typing import List
5
6 try:
7     import win32com.client
8     from win32com.client import dynamic
9 except Exception as e:
10     win32com = None
11     dynamic = None
12     WIN32_IMPORT_ERROR = e
13 else:
14     WIN32_IMPORT_ERROR = None
15
16 CM_TO_PT = 28.3464567
```

```

17
18 WD_ALIGN_CENTER = 1
19 WD_WITHIN_TABLE = 12
20 WD_HEADER_FOOTER_PRIMARY = 1
21 WD_LINE_SPACE_SINGLE = 0
22 WD_STATISTIC_PAGES = 2
23 WD_ACTIVE_END_PAGE_NUMBER = 3
24
25 WD_COLOR_RED = 255
26 WD_FIELD_PAGE = 33
27
28
29 def pt_to_cm(pt):
30     return pt / CM_TO_PT
31
32 @dataclass
33 class CheckIssue:
34     rule: str
35     severity: str
36     location: str
37     message: str
38     priority: int = 3
39
40
41 @dataclass
42 class CheckReport:
43     issues: List[CheckIssue] = field(default_factory=list)
44
45     def to_text(self):
46         if not self.issues:
47             return "❑ Ошибок не найдено."
48
49         self.issues.sort(key=lambda x: (x.priority, x.location))
50
51         lines = []
52         for i, issue in enumerate(self.issues, 1):
53             lines.append(f"❑ Ошибка #{i}")
54             lines.append(f"❑ Раздел: {issue.rule}")
55             lines.append(f"❑ Где: {issue.location}")
56             lines.append(f"❑ Что нужно исправить: {issue.message}")
57             lines.append("")
58         return "\n".join(lines)
59
60 class WordMethodicalChecker:
61     def __init__(self, visible: bool = False, mark_document: bool = True):
62         if dynamic is None:
63             raise RuntimeError(f"pywin32 не установлен: {WIN32_IMPORT_ERROR}")
64         self.word = None
65         self.visible = visible
66         self.mark_document = mark_document
67
68     def __enter__(self):
69         try:

```

```

70         self.word = dynamic.Dispatch("Word.Application")
71
72         self.word.DisplayAlerts = 0
73
74         try:
75             self.word.Visible = self.visible
76         except Exception as e:
77             print(f"Warning: Не удалось установить Word.Visible: {e}")
78
79         except Exception as e:
80             raise RuntimeError(f"Не удалось запустить Microsoft Word через COM: {e}")
81
82         return self
83
84     def __exit__(self, exc_type, exc_val, exc_tb):
85         if self.word:
86             self.word.Quit(False)
87
88     def open_document(self, path, readonly=True):
89         return self.word.Documents.Open(os.path.abspath(path), ReadOnly=
            readonly)
90
91     def check(self, path):
92         report = CheckReport()
93
94         checked_path = None
95         open_path = path
96         readonly = True
97
98         if self.mark_document:
99             base, ext = os.path.splitext(path)
100             checked_path = base + "_checked" + ext
101             shutil.copy(path, checked_path)
102             open_path = checked_path
103             readonly = False
104
105         doc = self.open_document(open_path, readonly=readonly)
106
107         try:
108             self._check_paragraphs(doc, report)
109             self._check_margins(doc, report)
110             self._check_page_numbers(doc, report)
111
112             if self.mark_document:
113                 doc.Save()
114         finally:
115             doc.Close(False)
116
117         return report, checked_path
118
119
120     def _mark_range(self, self, doc, start_par, end_par, message):
121         if not self.mark_document:

```

```

122         return
123
124     try:
125         start = doc.Paragraphs(start_par).Range.Start
126         end = doc.Paragraphs(end_par).Range.End
127         rng = doc.Range(start, end)
128
129         rng.Font.Color = WD_COLOR_RED
130         doc.Comments.Add(rng, message)
131     except Exception:
132         pass
133
134     def _mark_page(self, doc, page, message):
135         if not self.mark_document:
136             return
137
138         try:
139             start = doc.GoTo(1, 1, page).Start
140             try:
141                 end = doc.GoTo(1, 1, page + 1).Start
142             except Exception:
143                 end = doc.Content.End
144
145             rng = doc.Range(start, end)
146             rng.Font.Color = WD_COLOR_RED
147             doc.Comments.Add(rng, message)
148         except Exception:
149             pass
150
151     def _group_pages(self, doc, report, pages, message, rule="3", priority=1)
152     :
153         pages = sorted(set(pages))
154         if not pages:
155             return
156
157         start = prev = pages[0]
158
159         for p in pages[1:] + [None]:
160             if p != prev + 1:
161                 loc = f"Стр. {start} - Стр. {prev}"
162
163                 report.issues.append(
164                     CheckIssue(
165                         rule=rule,
166                         severity="ERROR",
167                         location=loc,
168                         message=message,
169                         priority=priority,
170                     )
171                 )
172
173                 for pg in range(start, prev + 1):
174                     self._mark_page(doc, pg, message)

```

```

175         if p is not None:
176             start = p
177         prev = p
178
179     def _check_paragraphs(self, doc, report):
180         font_pages = []
181         size_pages = []
182         indent_pages = []
183         spacing_pages = []
184
185         for i in range(1, doc.Paragraphs.Count + 1):
186             p = doc.Paragraphs(i)
187             text = (p.Range.Text or "").strip()
188
189             if not text or len(text) < 3:
190                 continue
191
192             try:
193                 if p.Range.Information(WD_WITHIN_TABLE):
194                     continue
195             except Exception:
196                 pass
197
198             try:
199                 style_name = str(p.Range.Style.NameLocal).lower()
200                 if "heading" in style_name or "заголол" in style_name:
201                     continue
202             except Exception:
203                 pass
204
205             page = p.Range.Information(WD_ACTIVE_END_PAGE_NUMBER)
206
207             indent = p.Range.ParagraphFormat.FirstLineIndent
208             if abs(indent) < 0.01:
209                 try:
210                     indent = p.Range.Style.ParagraphFormat.FirstLineIndent
211                 except Exception:
212                     pass
213
214             indent_cm = pt_to_cm(indent)
215             if abs(indent_cm - 1.25) > 0.05:
216                 indent_pages.append(page)
217
218             font = str(p.Range.Font.Name).strip()
219             if font != "Times New Roman":
220                 font_pages.append(page)
221
222             try:
223                 size = float(p.Range.Font.Size)
224             except Exception:
225                 size = 16.0
226
227             if size == 9999999.0:
228                 real_bad_size_found = False

```

```

229
230         try:
231             words = p.Range.Words
232             for j in range(1, words.Count + 1):
233                 w = words(j)
234                 w_text = (w.Text or "").strip()
235
236                 if not w_text:
237                     continue
238
239                 if w_text in {".", "/", ";", ":", "-", "—", "(",
240                                , ")", "[", "]", "{", "}", "/", "\\"}:
241                     continue
242
243                 try:
244                     w_size = float(w.Font.Size)
245                 except Exception:
246                     continue
247
248                 if w_size == 9999999.0:
249                     continue
250
251                 if abs(w_size - 16) > 0.2:
252                     real_bad_size_found = True
253                     break
254
255                 if real_bad_size_found:
256                     size_pages.append(page)
257
258             except Exception:
259                 pass
260         else:
261             if abs(size - 16) > 0.2:
262                 size_pages.append(page)
263
264         spacing = int(p.Range.ParagraphFormat.LineSpacingRule)
265         if spacing != WD_LINE_SPACE_SINGLE:
266             spacing_pages.append(page)
267
268     self._group_pages(
269         doc,
270         report,
271         font_pages,
272         (
273             "В основном тексте используется неверный шрифт. "
274             "Необходимо установить шрифт Times New Roman для всего
275             основного текста документа."
276         ),
277         rule="3",
278         priority=1,
279     )
280
281     self._group_pages(
282         doc,

```

```

281         report,
282         size_pages,
283         (
284             "В основном тексте используется неверный размер шрифта. "
285             "Необходимо установить размер шрифта 16 pt для всего
                основного текста документа."
286         ),
287         rule="3",
288         priority=1,
289     )
290
291     self._group_pages(
292         doc,
293         report,
294         indent_pages,
295         (
296             "Обнаружен неверный абзацный отступ первой строки. "
297             "Необходимо установить отступ первой строки 1,25 см."
298         ),
299         rule="3",
300         priority=1,
301     )
302
303     self._group_pages(
304         doc,
305         report,
306         spacing_pages,
307         (
308             "Обнаружен неверный межстрочный интервал. "
309             "Необходимо установить одинарный межстрочный интервал для
                основного текста документа."
310         ),
311         rule="3",
312         priority=1,
313     )
314
315     def _check_margins(self, doc, report):
316         pages = []
317
318         for i in range(1, doc.Sections.Count + 1):
319             sec = doc.Sections(i)
320             ps = sec.PageSetup
321
322             if (
323                 abs(pt_to_cm(ps.TopMargin) - 3.0) > 0.05 or
324                 abs(pt_to_cm(ps.BottomMargin) - 2.25) > 0.05 or
325                 abs(pt_to_cm(ps.LeftMargin) - 2.2) > 0.05 or
326                 abs(pt_to_cm(ps.RightMargin) - 2.3) > 0.05
327             ):
328                 pages.append(i)
329
330     self._group_pages(
331         doc,
332         report,

```

```

333         pages,
334         (
335             "Обнаружены неверные размеры полей страницы. "
336             "Необходимо установить поля документа: верхнее — 3 см, "
337             "нижнее — 2,25 см, левое — 2,2 см, правое — 2,3 см."
338         ),
339         rule="3",
340         priority=1,
341     )
342
343     def _check_page_numbers(self, doc, report):
344         total = doc.ComputeStatistics(WD_STATISTIC_PAGES)
345
346         found_top = False
347         found_bottom = False
348         issues = []
349
350         for i in range(1, doc.Sections.Count + 1):
351             sec = doc.Sections(i)
352
353             header = sec.Headers(WD_HEADER_FOOTER_PRIMARY)
354             footer = sec.Footers(WD_HEADER_FOOTER_PRIMARY)
355
356             for f in header.Range.Fields:
357                 if f.Type == WD_FIELD_PAGE:
358                     found_top = True
359                     try:
360                         para = f.Result.Paragraphs(1)
361                         if int(para.Alignment) != WD_ALIGN_CENTER:
362                             issues.append(
363                                 "Номер страницы расположен в верхнем
364                                 колонтитуле, но выровнен не по центру. "
365                                 "Необходимо выровнять номер страницы строго
366                                 по центру."
367                             )
368                     except Exception:
369                         issues.append(
370                             "Не удалось определить выравнивание номера
371                             страницы. "
372                             "Проверьте, что номер страницы расположен сверху
373                             и строго по центру."
374                         )
375
376             if footer.PageNumbers.Count > 0:
377                 found_bottom = True
378
379         loc = f"Стр. 1 - Стр. {total}"
380
381         if not found_top and not found_bottom:
382             message = (
383                 "Нумерация страниц отсутствует. "
384                 "Необходимо добавить номер страницы в верхней части страницы
385                 и выровнять его по центру."
386             )

```

```

382         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
383         self._mark_page(doc, 1, message)
384         return
385
386     if not found_top and found_bottom:
387         message = (
388             "Номер страницы расположен внизу страницы. "
389             "По требованиям номер страницы должен находиться вверху
390             страницы и быть выровнен по центру."
391         )
392         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
393         self._mark_page(doc, 1, message)
394         return
395
396     if issues:
397         message = " ".join(issues)
398         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
399         self._mark_page(doc, 1, message)

```

email

```

1  import smtplib
2  import ssl
3  from email.mime.multipart import MIMEMultipart
4  from email.mime.base import MIMEBase
5  from email.mime.text import MIMEText
6  from email import encoders
7  from email.header import Header
8  import os
9  from typing import List
10 from fastapi import HTTPException
11 import logging
12
13 from app.core.email_config import (
14     SMTP_HOST, SMTP_PORT, SMTP_USER,
15     SMTP_PASSWORD, SMTP_EMAIL,
16     SMTP_FROM_EMAIL, SMTP_FROM_NAME
17 )
18
19 logger = logging.getLogger(__name__)
20
21
22 def send_email_with_attachments(
23     to_email: str,
24     subject: str,
25     body: str,
26     file_paths: List[str],
27     file_names: List[str] = None
28 ) -> bool:
29     """
30     Отправка email с вложениями
31     """
32     if not SMTP_USER or not SMTP_PASSWORD:
33         raise HTTPException(
34             status_code=500,

```

```

35         detail="SMTP настройки не заданы. Проверьте .env файл"
36     )
37
38     try:
39         msg = MIMEMultipart()
40         msg['From'] = SMTP_USER
41         msg['To'] = to_email
42         msg['Subject'] = Header(subject, 'utf-8').encode()
43
44         full_body = f"От: {SMTP_FROM_NAME} <{SMTP_FROM_EMAIL}>\n\n{body}"
45         msg.attach(MIMEText(full_body, 'plain', 'utf-8'))
46
47         for idx, file_path in enumerate(file_paths):
48             if not os.path.exists(file_path):
49                 logger.warning(f"Файл не найден: {file_path}")
50                 continue
51
52             with open(file_path, "rb") as attachment:
53                 part = MIMEBase('application', 'octet-stream')
54                 part.set_payload(attachment.read())
55                 encoders.encode_base64(part)
56
57                 if file_names and idx < len(file_names):
58                     filename = file_names[idx]
59                 else:
60                     filename = os.path.basename(file_path)
61
62                 part.add_header(
63                     'Content-Disposition',
64                     'attachment',
65                     filename=Header(filename, 'utf-8').encode()
66                 )
67                 msg.attach(part)
68
69         if SMTP_PORT == 465:
70             context = ssl.create_default_context()
71             with smtplib.SMTP_SSL(SMTP_HOST, SMTP_PORT, context=context) as server:
72                 server.login(SMTP_USER, SMTP_PASSWORD)
73                 server.send_message(msg)
74         else:
75             with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
76                 server.starttls()
77                 server.login(SMTP_USER, SMTP_PASSWORD)
78                 server.send_message(msg)
79
80         logger.info(f"Письмо успешно отправлено на {to_email}")
81         return True
82
83     except smtplib.SMTPAuthenticationError as e:
84         logger.error(f"Ошибка авторизации: {e}")
85         raise HTTPException(
86             status_code=500,

```

```

87         detail=f"Ошибка авторизации. Проверьте пароль приложения. {str(e)}
            }"
88     )
89     except Exception as e:
90         logger.error(f"Ошибка: {e}")
91         raise HTTPException(status_code=500, detail=f"Ошибка отправки: {str(e)}")
92
93
94     def send_to_rio(
95         user_fio: str,
96         user_email: str,
97         comment: str,
98         file_paths: List[str],
99         file_names: List[str]
100     ) -> bool:
101         """
102         Отправка файлов в РИО
103         """
104         subject = f"Новые методические материалы от {user_fio}"
105
106         body = f"""
107         Уважаемые сотрудники РИО!
108
109         Пользователь {user_fio} ({user_email}) отправил на проверку следующие файлы:
110
111         {chr(10).join(f'- {name}' for name in file_names)}
112
113         Комментарий отправителя:
114         {comment if comment else 'Без комментария'}
115
116         ---
117         Это автоматическое сообщение, пожалуйста, не отвечайте на него.
118
119         Сгенерировано системой проверки файлов.
120         """
121
122         return send_email_with_attachments(
123             to_email=SMTP_RIO_EMAIL,
124             subject=subject,
125             body=body,
126             file_paths=file_paths,
127             file_names=file_names
128         )

```

authService

```

1 from datetime import datetime, timezone
2
3 from fastapi import HTTPException, Request, status
4 from sqlalchemy.orm import Session
5
6 from app.core.security import (
7     create_access_token,
8     create_refresh_token,

```

```

9     get_refresh_token_expiry,
10    get_refresh_token_hash,
11    hash_password,
12    verify_password,
13 )
14 from app.models.department import Department
15 from app.models.faculty import Faculty
16 from app.models.refresh_token import RefreshToken
17 from app.models.user import User
18
19 ALLOWED_ROLES = {
20     "Доцент",
21     "Профессор",
22     "Старший преподаватель",
23     "Преподаватель",
24     "Аспирант",
25 }
26
27
28 def register_user(db: Session, fio: str, email: str, password: str,
29                  faculty_code: str, department_name: str, role: str) -> User:
30     existing_user = db.query(User).filter(User.email == email).first()
31     if existing_user:
32         raise HTTPException(status_code=400, detail="Пользователь с таким
33                               email уже существует")
34
35     if role not in ALLOWED_ROLES:
36         raise HTTPException(status_code=400, detail="Недопустимая роль")
37
38     faculty = db.query(Faculty).filter(Faculty.faculty_code == faculty_code).
39         first()
40     if not faculty:
41         raise HTTPException(status_code=404, detail="Факультет с таким кодом
42                               не найден")
43
44     department = db.query(Department).filter(Department.department_name ==
45         department_name).first()
46     if not department:
47         raise HTTPException(status_code=404, detail="Кафедра с таким
48                               названием не найдена")
49
50     if department.id_faculty != faculty.id_faculty:
51         raise HTTPException(status_code=400, detail="Кафедра не принадлежит
52                               указанному факультету")
53
54     user = User(
55         fio=fio,
56         email=email,
57         password_hash=hash_password(password),
58         role=role,
59         id_faculty=faculty.id_faculty,
60         id_department=department.id_department,
61     )

```

```

56 db.add(user)
57 db.commit()
58 db.refresh(user)
59 return user
60
61
62 def login_user(db: Session, email: str, password: str, request: Request) ->
tuple[str, str]:
63     user = db.query(User).filter(User.email == email).first()
64     if not user or not verify_password(password, user.password_hash):
65         raise HTTPException(
66             status_code=status.HTTP_401_UNAUTHORIZED,
67             detail="Неверный email или пароль"
68         )
69
70     access_token = create_access_token(user_id=user.id_user, email=user.email
)
71     refresh_token = create_refresh_token()
72
73     refresh_record = RefreshToken(
74         token_hash=get_refresh_token_hash(refresh_token),
75         expires_at=get_refresh_token_expiry(),
76         revoked=False,
77         user_agent=request.headers.get("user-agent"),
78         ip_address=request.client.host if request.client else None,
79         id_user=user.id_user,
80     )
81
82     db.add(refresh_record)
83     db.commit()
84
85     return access_token, refresh_token
86
87
88 def refresh_user_token(db: Session, refresh_token: str) -> str:
89     token_hash = get_refresh_token_hash(refresh_token)
90
91     token_record = (
92         db.query(RefreshToken)
93         .filter(RefreshToken.token_hash == token_hash)
94         .first()
95     )
96
97     if not token_record:
98         raise HTTPException(status_code=401, detail="Refresh token не найден")
99
100     if token_record.revoked:
101         raise HTTPException(status_code=401, detail="Refresh token отозван")
102
103     expires_at = token_record.expires_at
104     if expires_at.tzinfo is None:
105         expires_at = expires_at.replace(tzinfo=timezone.utc)
106

```

```

107     if expires_at < datetime.now(timezone.utc):
108         raise HTTPException(status_code=401, detail="Refresh token истёк")
109
110     user = db.query(User).filter(User.id_user == token_record.id_user).first()
111
112     if not user:
113         raise HTTPException(status_code=401, detail="Пользователь не найден")
114
115     return create_access_token(user_id=user.id_user, email=user.email)
116
117 def logout_user(db: Session, refresh_token: str):
118     token_hash = get_refresh_token_hash(refresh_token)
119
120     token_record = (
121         db.query(RefreshToken)
122         .filter(RefreshToken.token_hash == token_hash)
123         .first()
124     )
125
126     if not token_record:
127         raise HTTPException(status_code=404, detail="Refresh token не найден")
128
129     token_record.revoked = True
130     db.commit()
131
132
133 def logout_all_user_sessions(db: Session, user_id: int):
134     tokens = db.query(RefreshToken).filter(
135         RefreshToken.id_user == user_id,
136         RefreshToken.revoked == False
137     ).all()
138
139     for token in tokens:
140         token.revoked = True
141
142     db.commit()

```

pdfReport

```

1 import os
2 from datetime import datetime
3
4 from reportlab.lib import colors
5 from reportlab.lib.pagesizes import A4
6 from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
7 from reportlab.lib.units import mm
8 from reportlab.pdfbase import pdfmetrics
9 from reportlab.pdfbase.ttfonts import TTFont
10 from reportlab.platypus import SimpleDocTemplate, Spacer, Paragraph, Table,
    TableStyle
11
12 from app.service.checker import CheckReport
13

```

```

14
15 def register_fonts():
16     fonts_dir = os.path.join("app", "fonts")
17
18     pdfmetrics.registerFont(TTFont("TimesNewRoman", os.path.join(fonts_dir, "
        times.ttf")))
19     pdfmetrics.registerFont(TTFont("TimesNewRoman-Bold", os.path.join(
        fonts_dir, "timesbd.ttf")))
20     pdfmetrics.registerFont(TTFont("TimesNewRoman-Italic", os.path.join(
        fonts_dir, "timesi.ttf")))
21     pdfmetrics.registerFont(TTFont("TimesNewRoman-BoldItalic", os.path.join(
        fonts_dir, "timesbi.ttf")))
22
23
24 def generate_error_report_pdf(report: CheckReport, source_filename: str,
    output_path: str) -> str:
25     os.makedirs(os.path.dirname(output_path), exist_ok=True)
26
27     register_fonts()
28
29     doc = SimpleDocTemplate(
30         output_path,
31         pagesize=A4,
32         leftMargin=15 * mm,
33         rightMargin=15 * mm,
34         topMargin=15 * mm,
35         bottomMargin=15 * mm,
36     )
37
38     styles = getSampleStyleSheet()
39
40     title_style = ParagraphStyle(
41         "TitleCustom",
42         parent=styles["Title"],
43         fontName="TimesNewRoman-Bold",
44         fontSize=16,
45         leading=20,
46         textColor=colors.HexColor("#1f2937"),
47         spaceAfter=8,
48     )
49
50     meta_style = ParagraphStyle(
51         "MetaCustom",
52         parent=styles["Normal"],
53         fontName="TimesNewRoman",
54         fontSize=10,
55         leading=13,
56         textColor=colors.HexColor("#4b5563"),
57         spaceAfter=4,
58     )
59
60     section_style = ParagraphStyle(
61         "SectionCustom",
62         parent=styles["Heading2"],

```

```

63     fontName="TimesNewRoman- Bold" ,
64     fontSize=12,
65     leading=16,
66     textColor=colors.HexColor( "#111827" ),
67     spaceBefore=8,
68     spaceAfter=6,
69 )
70
71 body_style = ParagraphStyle(
72     "BodyCustom" ,
73     parent=styles[ "Normal" ],
74     fontName="TimesNewRoman" ,
75     fontSize=9,
76     leading=12,
77     textColor=colors.black ,
78 )
79
80 table_header_style = ParagraphStyle(
81     "TableHeader" ,
82     parent=styles[ "Normal" ],
83     fontName="TimesNewRoman- Bold" ,
84     fontSize=9,
85     leading=11,
86     textColor=colors.HexColor( "#111827" ),
87     alignment=1,
88 )
89
90 table_cell_style = ParagraphStyle(
91     "TableCell" ,
92     parent=styles[ "Normal" ],
93     fontName="TimesNewRoman" ,
94     fontSize=8.5,
95     leading=10.5,
96     textColor=colors.black ,
97     wordWrap="CJK" ,
98 )
99
100 table_cell_center_style = ParagraphStyle(
101     "TableCellCenter" ,
102     parent=table_cell_style ,
103     alignment=1, # center
104 )
105
106 elements = []
107
108 errors_count = sum(1 for issue in report.issues if issue.severity == "
109     ERROR")
110
111 warnings_count = sum(1 for issue in report.issues if issue.severity == "
112     WARNING")
113
114 if errors_count > 0:
115     status_text = "Статус: документ содержит критические ошибки"
116     status_color = "#991b1b"
117 elif warnings_count > 0:

```

```

115     status_text = "Статус: документ содержит предупреждения"
116     status_color = "#92400e"
117 else:
118     status_text = "Статус: документ соответствует требованиям"
119     status_color = "#166534"
120
121 status_style = ParagraphStyle(
122     "StatusCustom",
123     parent=styles["Normal"],
124     fontName="TimesNewRoman-Bold",
125     fontSize=11,
126     leading=14,
127     textColor=colors.HexColor(status_color),
128     spaceAfter=8,
129 )
130
131 elements.append(Paragraph("Отчёт о проверке", title_style))
132 elements.append(Paragraph(f"Файл: {source_filename}", meta_style))
133 elements.append(Paragraph(f"Дата проверки: {datetime.now().strftime('%d.%m.%Y %H:%M:%S')}", meta_style))
134 elements.append(Paragraph(f"Ошибки: {errors_count} | Предупреждения: {warnings_count}", meta_style))
135 elements.append(Paragraph(status_text, status_style))
136 elements.append(Spacer(1, 6))
137
138 if not report.issues:
139     elements.append(Paragraph("Ошибки не найдены", section_style))
140     elements.append(Paragraph("Документ успешно прошёл автоматическую проверку оформления.", body_style))
141 else:
142     elements.append(Paragraph("Найденные замечания", section_style))
143
144 table_data = [
145     [
146         Paragraph("#", table_header_style),
147         Paragraph("Раздел", table_header_style),
148         Paragraph("Тип", table_header_style),
149         Paragraph("Где", table_header_style),
150         Paragraph("Описание", table_header_style),
151     ]
152 ]
153
154 for idx, issue in enumerate(report.issues, start=1):
155     severity_text = "Ошибка" if issue.severity == "ERROR" else "Предупреждение"
156
157     table_data.append([
158         Paragraph(str(idx), table_cell_center_style),
159         Paragraph(str(issue.rule), table_cell_center_style),
160         Paragraph(severity_text, table_cell_center_style),
161         Paragraph(issue.location, table_cell_style),
162         Paragraph(issue.message, table_cell_style),
163     ])
164

```

```

165     table = Table(
166         table_data,
167         colWidths=[10 * mm, 18 * mm, 26 * mm, 52 * mm, 64 * mm],
168         repeatRows=1,
169     )
170
171     table.setStyle(TableStyle([
172         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#e5e7eb")),
173         ("TEXTCOLOR", (0, 0), (-1, 0), colors.HexColor("#111827")),
174         ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#cbd5e1")),
175         ("VALIGN", (0, 0), (-1, -1), "TOP"),
176         ("ALIGN", (0, 0), (2, -1), "CENTER"),
177         ("BACKGROUND", (0, 1), (-1, -1), colors.white),
178         ("TOPPADDING", (0, 0), (-1, -1), 4),
179         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
180         ("LEFTPADDING", (0, 0), (-1, -1), 4),
181         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
182     ]))
183
184     for row_idx, issue in enumerate(report.issues, start=1):
185         if issue.severity == "ERROR":
186             table.setStyle(TableStyle([
187                 ("BACKGROUND", (2, row_idx), (2, row_idx), colors.
188                     HexColor("#fee2e2")),
189                 ("TEXTCOLOR", (2, row_idx), (2, row_idx), colors.HexColor(
190                     "#991b1b")),
191             ]))
192         elif issue.severity == "WARNING":
193             table.setStyle(TableStyle([
194                 ("BACKGROUND", (2, row_idx), (2, row_idx), colors.
195                     HexColor("#fef3c7")),
196                 ("TEXTCOLOR", (2, row_idx), (2, row_idx), colors.HexColor(
197                     "#92400e")),
198             ]))
199
200     elements.append(table)
201
202     doc.build(elements)
203     return output_path

```

titlePageGenerator

```

1  import os
2  import tempfile
3  import re
4
5  from uuid import uuid4
6
7  from docx import Document
8  from docx.enum.section import WD_SECTION_START
9  from docx.enum.text import WD_ALIGN_PARAGRAPH
10 from docx.oxml.ns import qn
11 from docx.shared import Pt, Cm
12
13

```

```

14 MINISTRY_NAME = "МИНОБРАЗОВАНИЯ РОССИИ"
15 UNIVERSITY_FULL_NAME = "Федеральное государственное бюджетное образовательное
    учреждение высшего образования"
16 UNIVERSITY_SHORT_NAME = "«Юго-Западный государственный университет» (ЮЗГУ)"
17
18
19 def set_font(run, size=16, bold=False):
20     run.font.name = "Times New Roman"
21     run._element.rPr.rFonts.set(qn("w: eastAsia"), "Times New Roman")
22     run.font.size = Pt(size)
23     run.bold = bold
24
25
26 def clear_paragraph(paragraph):
27     p = paragraph._element
28     for child in list(p):
29         p.remove(child)
30
31
32 def add_paragraph(
33     container,
34     text: str = "",
35     align: int = WD_ALIGN_PARAGRAPH.LEFT,
36     size: int = 16,
37     bold: bool = False,
38     space_before: int = 0,
39     space_after: int = 0,
40     line_spacing: float = 1.0,
41 ):
42     p = container.add_paragraph()
43     p.alignment = align
44     p.paragraph_format.space_before = Pt(space_before)
45     p.paragraph_format.space_after = Pt(space_after)
46     p.paragraph_format.line_spacing = line_spacing
47
48     run = p.add_run(text)
49     set_font(run, size=size, bold=bold)
50     return p
51
52
53 def add_empty_paragraphs(container, count: int, size: int = 16):
54     for _ in range(count):
55         p = container.add_paragraph()
56         p.paragraph_format.space_before = Pt(0)
57         p.paragraph_format.space_after = Pt(0)
58         p.paragraph_format.line_spacing = 1.0
59         run = p.add_run("")
60         set_font(run, size=size)
61
62
63 def normalize_department_name(name: str) -> str:
64     name = (name or "").strip()
65     lower_name = name.lower()
66     if lower_name.startswith("кафедра "):

```

```

67         return name[8:].strip()
68     return name
69
70
71 def set_footer_text(footer, text: str, size: int = 16):
72     paragraph = footer.paragraphs[0]
73     clear_paragraph(paragraph)
74     paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
75     paragraph.paragraph_format.space_before = Pt(0)
76     paragraph.paragraph_format.space_after = Pt(0)
77     paragraph.paragraph_format.line_spacing = 1.0
78
79     run = paragraph.add_run(text)
80     set_font(run, size=size, bold=False)
81
82
83 def set_first_page_footer(section, text: str, size: int = 16):
84     section.different_first_page_header_footer = True
85     set_footer_text(section.first_page_footer, text, size=size)
86     set_footer_text(section.footer, text, size=size)
87
88
89 def clear_section_footer(section):
90     section.different_first_page_header_footer = True
91     clear_paragraph(section.first_page_footer.paragraphs[0])
92     clear_paragraph(section.footer.paragraphs[0])
93
94
95 def format_tutorial_title(title: str) -> str:
96     title = (title or "").strip()
97     if not title:
98         return title
99     return title[:1].upper() + title[1:].lower()
100
101
102 def ensure_a_value_prefix(a_value: str) -> str:
103     value = (a_value or "").strip()
104     if not value:
105         return "A __"
106
107     upper_value = value.upper()
108     if upper_value.startswith("A "):
109         return value
110     if upper_value.startswith("A"):
111         return f"A {value[2:].strip()}"
112     if upper_value == "A" or upper_value == "A ":
113         return "A __"
114
115     return f"A {value}"
116
117
118 def format_author_name_for_biblio(author_name: str) -> str:
119     """
120     Поддерживает:

```

```

121 - Аникина Елена Ивановна -> Е. И. Аникина
122 - Аникина Е. И. -> Е. И. Аникина
123 - Аникина Е.И. -> Е. И. Аникина
124 - Е. И. Аникина -> Е. И. Аникина
125 - Е.И. Аникина -> Е. И. Аникина
126 """
127 text = (author_name or "").strip()
128 if not text:
129     return text
130
131 text = re.sub(r"\s+", " ", text)
132
133 m = re.match(r"^\s*([А-ЯА-З])\.\s*([А-ЯА-З])\.\s*([А-Яа-яА-За-з\-\-]+)$", text)
134 if m:
135     n1, n2, surname = m.groups()
136     return f"{n1}. {n2}. {surname}"
137
138 m = re.match(r"^\s*([А-ЯА-З])\.\s*([А-ЯА-З])\.\s*([А-Яа-яА-За-з\-\-]+)$", text)
139 if m:
140     n1, n2, surname = m.groups()
141     return f"{n1}. {n2}. {surname}"
142
143 parts = text.split()
144
145 if len(parts) == 3 and all(len(p) > 1 for p in parts):
146     surname, name, patronymic = parts
147     return f"{name[0]}. {patronymic[0]}. {surname}"
148
149 if len(parts) == 3 and len(parts[0]) > 1:
150     surname = parts[0]
151     p2 = parts[1].replace(".", "")
152     p3 = parts[2].replace(".", "")
153     if len(p2) == 1 and len(p3) == 1:
154         return f"{p2}. {p3}. {surname}"
155
156 return text
157
158
159 def generate_title_page_docx(
160     manual_title: str,
161     discipline_name: str,
162     audience: str,
163     direction_code: str,
164     direction_name: str,
165     department_name: str,
166     city: str,
167     year: int,
168     udk: str,
169     compiler_name: str,
170     reviewers: list,
171     description: str,
172 ) -> str:
173     output_dir = os.path.join(tempfile.gettempdir(), "title_pages")

```

```

174 os.makedirs(output_dir, exist_ok=True)
175
176 filename = f"title_page_{uuid4().hex}.docx"
177 output_path = os.path.join(output_dir, filename)
178
179 doc = Document()
180
181 section = doc.sections[0]
182 section.top_margin = Cm(3)
183 section.bottom_margin = Cm(2.25)
184 section.left_margin = Cm(2.2)
185 section.right_margin = Cm(2.3)
186 section.footer_distance = Cm(1.3)
187
188 normal_style = doc.styles["Normal"]
189 normal_style.font.name = "Times New Roman"
190 normal_style._element.rPr.rFonts.set(qn("w: eastAsia"), "Times New Roman")
191 normal_style.font.size = Pt(16)
192
193 clean_department_name = normalize_department_name(department_name)
194
195 add_paragraph(doc, MINISTRY_NAME, align=WD_ALIGN_PARAGRAPH.CENTER, size
196               =16)
197 add_paragraph(doc, UNIVERSITY_FULL_NAME, align=WD_ALIGN_PARAGRAPH.CENTER,
198               size=16)
199 add_paragraph(doc, UNIVERSITY_SHORT_NAME, align=WD_ALIGN_PARAGRAPH.CENTER
200               , size=16)
201 add_paragraph(
202     doc,
203     f"Кафедра {clean_department_name}",
204     align=WD_ALIGN_PARAGRAPH.CENTER,
205     size=16,
206 )
207
208 add_empty_paragraphs(doc, 1)
209
210 add_paragraph(
211     doc,
212     "УТВЕРЖДАЮ",
213     align=WD_ALIGN_PARAGRAPH.RIGHT,
214     size=16,
215     bold=True,
216 )
217 add_paragraph(
218     doc,
219     "Заведующий кафедрой _____",
220     align=WD_ALIGN_PARAGRAPH.RIGHT,
221     size=16,
222 )
223 add_paragraph(
224     doc,
225     "_____",
226     align=WD_ALIGN_PARAGRAPH.RIGHT,
227     size=16,

```

```

225 )
226 add_paragraph(
227     doc,
228     "_____ 20 ____ г. ",
229     align=WD_ALIGN_PARAGRAPH.RIGHT,
230     size=16,
231 )
232
233 add_empty_paragraphs(doc, 4)
234
235 add_paragraph(
236     doc,
237     manual_title.upper(),
238     align=WD_ALIGN_PARAGRAPH.CENTER,
239     size=16,
240     bold=True,
241     space_after=6,
242 )
243
244 add_paragraph(
245     doc,
246     (
247         f"Методические указания для выполнения лабораторной работы "
248         f"по дисциплине «{discipline_name}» для {audience} "
249         f"направления подготовки {direction_code} {direction_name}"
250     ),
251     align=WD_ALIGN_PARAGRAPH.CENTER,
252     size=16,
253     line_spacing=1.0,
254 )
255
256 add_paragraph(
257     doc,
258     f"{city} - {year}",
259     align=WD_ALIGN_PARAGRAPH.CENTER,
260     size=16,
261     space_before=210,
262 )
263
264 new_section = doc.add_section(WD_SECTION_START.NEW_PAGE)
265 new_section.top_margin = Cm(3)
266 new_section.bottom_margin = Cm(2.25)
267 new_section.left_margin = Cm(2.2)
268 new_section.right_margin = Cm(2.3)
269 new_section.footer_distance = Cm(1.3)
270 clear_section_footer(new_section)
271
272 add_paragraph(doc, f"УДК {udk}", size=16, space_after=0)
273 add_paragraph(doc, f"Составитель: {compiler_name}", size=16, space_after=6)
274
275 reviewers_title = "Рецензенты:" if len(reviewers) > 1 else "Рецензент:"
276 add_paragraph(doc, reviewers_title, size=16, space_after=0)
277

```

```

278     for reviewer in reviewers:
279         add_paragraph(
280             doc,
281             f"{reviewer.degree} {reviewer.fio}",
282             size=16,
283             space_after=0,
284         )
285     add_empty_paragraphs(doc, 1)
286
287     reviewers_fio = [r.fio for r in reviewers]
288     if len(reviewers_fio) == 1:
289         reviewers_str = reviewers_fio[0]
290     else:
291         reviewers_str = f"{'', ' '.join(reviewers_fio[:-1])} и {reviewers_fio[-1]}"
292
293     add_paragraph(
294         doc,
295         (
296             f"{manual_title}: методические указания для выполнения
297             лабораторной работы "
298             f"по дисциплине «{discipline_name}» для {audience} "
299             f"направления подготовки {direction_code} {direction_name}/ "
300             f"Юро-Зап. гос. ун-т; сост. {compiler_name}; рец. {reviewers_str}
301             "). "
302             f"{city}, {year}. 19 с."
303         ),
304         size=16,
305         line_spacing=1.0,
306         space_after=6,
307     )
308
309     add_paragraph(
310         doc,
311         (
312             f"Составлены в соответствии с федеральным государственным
313             образовательным стандартом "
314             f"высшего образования направления подготовки {direction_code} {
315             direction_name} "
316             f"и на основании учебного плана направления подготовки {
317             direction_code} {direction_name}."
318         ),
319         size=16,
320         line_spacing=1.0,
321         space_after=6,
322     )
323
324     add_paragraph(
325         doc,
326         description,
327         size=16,
328         line_spacing=1.0,
329         space_after=6,
330     )

```

```

326
327 add_paragraph(
328     doc,
329     (
330         f"Предназначены для {audience}, обучающихся направления
              подготовки "
331         f"{direction_code} {direction_name} "
332         f"(профиль «Разработка программноинформационных систем») всех
              форм обучения."
333     ),
334     size=16,
335     line_spacing=1.0,
336     space_after=8,
337 )
338
339 add_paragraph(
340     doc,
341     "Текст печатается в авторской редакции",
342     size=16,
343     space_after=12,
344 )
345
346 add_paragraph(
347     doc,
348     (
349         "Подписано в печать _____. Формат 60x84 1/16. "
350         "Усл. печ. л. _____. Уч.-изд. л. 2,0. "
351         "Тираж ____ экз. Заказ _____. Бесплатно."
352     ),
353     size=16,
354     line_spacing=1.0,
355     space_after=6,
356 )
357
358 add_paragraph(
359     doc,
360     "Юго-Западный государственный университет.",
361     size=16,
362     line_spacing=1.0,
363     space_after=0,
364 )
365
366 add_paragraph(
367     doc,
368     "305040, г. Курск, ул. 50 лет Октября, 94.",
369     size=16,
370     line_spacing=1.0,
371     space_after=0,
372 )
373
374 doc.save(output_path)
375 return output_path
376
377

```

```

378 def generate_tutorial_title_page_docx(
379     author_name: str,
380     tutorial_title: str,
381     city: str,
382     year: int,
383     reviewers: list,
384     a_value: str,
385     isbn: str,
386     directions: list,
387     udk: str,
388     bbk: str,
389     description: str,
390 ) -> str:
391     output_dir = os.path.join(tempfile.gettempdir(), "title_pages")
392     os.makedirs(output_dir, exist_ok=True)
393
394     filename = f"tutorial_title_page_{uuid4().hex}.docx"
395     output_path = os.path.join(output_dir, filename)
396
397     doc = Document()
398
399     section = doc.sections[0]
400     section.top_margin = Cm(3)
401     section.bottom_margin = Cm(2.25)
402     section.left_margin = Cm(2.2)
403     section.right_margin = Cm(2.3)
404
405     normal_style = doc.styles["Normal"]
406     normal_style.font.name = "Times New Roman"
407     normal_style._element.rPr.rFonts.set(qn("w: eastAsia"), "Times New Roman")
408     normal_style.font.size = Pt(16)
409
410     title_for_biblio = format_tutorial_title(tutorial_title)
411     a_value_formatted = ensure_a_value_prefix(a_value)
412     author_name_biblio = format_author_name_for_biblio(author_name)
413
414     add_paragraph(doc, "МИНОБРАЗОВАНИЯ РОССИИ", align=WD_ALIGN_PARAGRAPH.CENTER,
415                    size=16)
416     add_paragraph(doc, "Федеральное государственное бюджетное", align=
417                    WD_ALIGN_PARAGRAPH.CENTER, size=16)
418     add_paragraph(doc, "образовательное учреждение высшего", align=
419                    WD_ALIGN_PARAGRAPH.CENTER, size=16)
420     add_paragraph(doc, "образования", align=WD_ALIGN_PARAGRAPH.CENTER, size
421                    =16)
422     add_paragraph(doc, "«Юго-Западный государственный университет»", align=
423                    WD_ALIGN_PARAGRAPH.CENTER, size=16)
424     add_paragraph(doc, "(ЮЗГУ)", align=WD_ALIGN_PARAGRAPH.CENTER, size=16)
425
426     add_empty_paragraphs(doc, 6)
427
428     add_paragraph(doc, author_name_biblio, align=WD_ALIGN_PARAGRAPH.CENTER,
429                    size=16)
430     add_empty_paragraphs(doc, 2)

```

```

426 add_paragraph(doc, tutorial_title.upper(), align=WD_ALIGN_PARAGRAPH.
      CENTER, size=16, bold=True)
427 add_empty_paragraphs(doc, 1)
428 add_paragraph(doc, "Учебное пособие", align=WD_ALIGN_PARAGRAPH.CENTER,
      size=16)
429 add_empty_paragraphs(doc, 6)
430
431 add_paragraph(
432     doc,
433     "Утверждено Учебно-методическим составом",
434     align=WD_ALIGN_PARAGRAPH.CENTER,
435     size=16,
436 )
437 add_paragraph(
438     doc,
439     "Юго-Западного государственного университета",
440     align=WD_ALIGN_PARAGRAPH.CENTER,
441     size=16,
442 )
443
444 add_paragraph(
445     doc,
446     f"{city} {year}",
447     align=WD_ALIGN_PARAGRAPH.CENTER,
448     size=16,
449     space_before=140,
450 )
451
452 new_section = doc.add_section(WD_SECTION_START.NEW_PAGE)
453 new_section.top_margin = Cm(3)
454 new_section.bottom_margin = Cm(2.25)
455 new_section.left_margin = Cm(2.2)
456 new_section.right_margin = Cm(2.3)
457 clear_section_footer(new_section)
458
459 reviewers_title = "Рецензенты:" if len(reviewers) > 1 else "Рецензент:"
460 add_paragraph(doc, reviewers_title, size=16, space_after=4)
461
462 for reviewer in reviewers:
463     text = f"{reviewer.degree_position} {reviewer.fio}"
464     if text.strip().lower().startswith("рецензенты:"):
465         text = text.split(":", 1)[1].strip()
466     if text.strip().lower().startswith("рецензент:"):
467         text = text.split(":", 1)[1].strip()
468
469     add_paragraph(
470         doc,
471         text,
472         size=16,
473         space_after=4,
474     )
475
476 add_empty_paragraphs(doc, 1)
477

```

```

478 add_paragraph(doc, author_name_biblio, size=16)
479 add_empty_paragraphs(doc, 1)
480
481 add_paragraph(
482     doc,
483     f"{a_value_formatted} {title_for_biblio}: учеб. пособие / {
         author_name_biblio};",
484     size=16,
485 )
486 add_paragraph(
487     doc,
488     f"Юго-Зап. гос. ун-т. - {city}, {year}. - ____ с. - Библиогр.: с. ____
         .",
489     size=16,
490 )
491 add_paragraph(doc, f"ISBN {isbn}", size=16, space_after=8)
492
493 add_paragraph(doc, description, size=16, line_spacing=1.0, space_after=8)
494
495 if len(directions) == 1:
496     directions_text = f"{directions[0].code} «{directions[0].faculty_name
         }»"
497 else:
498     directions_text = ", ".join(
499         [f"{item.code} «{item.faculty_name}»" for item in directions
500        [:-1]]
501     ) + f" и {directions[-1].code} «{directions[-1].faculty_name}»"
502
503 add_paragraph(
504     doc,
505     f"Предназначено для студентов и магистрантов укрупненных групп
         направлений подготовки {directions_text}.",
506     size=16,
507     line_spacing=1.0,
508     space_after=8,
509 )
510
511 add_paragraph(doc, f"УДК {udk}", size=16)
512 add_paragraph(doc, f"ББК {bbk}", size=16, space_after=8)
513
514 add_paragraph(doc, f"ISBN {isbn}", size=16, space_after=8)
515
516 add_paragraph(doc, f"© Юго-Западный государственный университет, {year}",
517     size=16)
518 add_paragraph(doc, f"© {author_name_biblio}, {year}", size=16)
519
520 doc.save(output_path)
521 return output_path
522
523 def generate_monograph_title_page_docx(
524     authors: list,
525     monograph_title: str,
526     city: str,

```

```

526         year: int,
527         udk: str,
528         bbk: str,
529         isbn: str,
530         description: str,
531     ) -> str:
532         """
533         Генерация титульного листа монографии
534         """
535         output_dir = os.path.join(tempfile.gettempdir(), "title_pages")
536         os.makedirs(output_dir, exist_ok=True)
537
538         filename = f"monograph_title_page_{uuid4().hex}.docx"
539         output_path = os.path.join(output_dir, filename)
540
541         doc = Document()
542
543         section = doc.sections[0]
544         section.top_margin = Cm(3)
545         section.bottom_margin = Cm(2.25)
546         section.left_margin = Cm(2.2)
547         section.right_margin = Cm(2.3)
548
549         normal_style = doc.styles["Normal"]
550         normal_style.font.name = "Times New Roman"
551         normal_style._element.rPr.rFonts.set(qn("w: eastAsia"), "Times New Roman")
552         normal_style.font.size = Pt(16)
553
554         authors_fio = [author.fio if hasattr(author, 'fio') else author for
555                        author in authors]
556
557         if len(authors_fio) == 1:
558             authors_biblio = authors_fio[0]
559         else:
560             authors_biblio = f"{'', '.join(authors_fio[:-1])}, {authors_fio[-1]}"
561
562         authors_str = ", ".join(authors_fio)
563
564         add_paragraph(doc, "МИНИСТЕРСТВО ОБРАЗОВАНИЯ РФ", align=
565                        WD_ALIGN_PARAGRAPH.CENTER, size=16)
566         add_paragraph(doc, "Государственное образовательное учреждение высшего
567                        профессионального образования",
568                        align=WD_ALIGN_PARAGRAPH.CENTER, size=16)
569         add_paragraph(doc, "«Юго-Западный государственный университет»", align=
570                        WD_ALIGN_PARAGRAPH.CENTER, size=16)
571
572         add_empty_paragraphs(doc, 4)
573
574         add_paragraph(doc, authors_str, align=WD_ALIGN_PARAGRAPH.CENTER, size=16,
575                        bold=True)
576
577         add_empty_paragraphs(doc, 2)

```

```

574 add_paragraph(doc, monograph_title.upper(), align=WD_ALIGN_PARAGRAPH.
      CENTER, size=16, bold=True)
575
576 add_empty_paragraphs(doc, 1)
577
578 add_paragraph(doc, "Монография", align=WD_ALIGN_PARAGRAPH.CENTER, size
      =16)
579
580 add_empty_paragraphs(doc, 6)
581
582 add_paragraph(doc, f"{city} - {year}", align=WD_ALIGN_PARAGRAPH.CENTER,
      size=16)
583
584 new_section = doc.add_section(WD_SECTION_START.NEW_PAGE)
585 new_section.top_margin = Cm(3)
586 new_section.bottom_margin = Cm(2.25)
587 new_section.left_margin = Cm(2.2)
588 new_section.right_margin = Cm(2.3)
589 clear_section_footer(new_section)
590
591 add_paragraph(doc, f"УДК {udk}", size=16, space_after=0)
592 add_paragraph(doc, f"ББК {bbk}", size=16, space_after=12)
593
594 add_paragraph(doc, "Рецензенты:", size=16, space_after=6)
595
596 add_empty_paragraphs(doc, 1)
597
598 biblio_text = f"{authors_biblio}. {monograph_title}: монография / {
      authors_biblio}; Юго-Западный гос. ун-т. - {city}, {year}. - ____ с. :
      ил. ____, табл. ____, Библиогр.: с. ____"
599 add_paragraph(doc, biblio_text, size=16, line_spacing=1.0, space_after=6)
600
601 add_paragraph(doc, f"ISBN {isbn}", size=16, space_after=12)
602
603 add_paragraph(doc, description, size=16, line_spacing=1.0, space_after
      =12)
604
605 add_paragraph(doc, f"УДК {udk}", size=16, space_after=0)
606 add_paragraph(doc, f"ББК {bbk}", size=16, space_after=12)
607
608 add_paragraph(doc, f"ISBN {isbn}", size=16, space_after=6)
609 add_paragraph(doc, f"© Юго-Западный государственный университет, {year}",
      size=16, space_after=0)
610 add_paragraph(doc, f"© {authors_biblio}, {year}", size=16)
611
612 doc.save(output_path)
613 return output_path

```

Место для диска